

Complete DSA Notes: Sorting in C++

From Beginner Basics to Interview Mastery

- [1. Sorting Basics](#)
 - [What is Sorting?](#)
 - [Why Do We Need Sorting?](#)
 - [Real-World Applications](#)
 - [Comparison vs Non-Comparison Sorting](#)
 - [Stable vs Unstable Sorting](#)
 - [In-Place vs Out-of-Place Sorting](#)
 - [Adaptive vs Non-Adaptive Sorting](#)
 - [Internal vs External Sorting](#)
 - [Time Complexity](#)
 - [Space Complexity](#)
 - [Best, Average, and Worst Case — Why All Three Matter](#)
- [2. Basic Sorting Algorithms](#)
 - [Bubble Sort](#)
 - [Selection Sort](#)
 - [Insertion Sort](#)
- [3. Important Efficient Sorting Algorithms](#)
 - [Merge Sort](#)
 - [Quick Sort](#)
 - [Heap Sort](#)
- [4. Non-Comparison Sorting](#)
 - [Counting Sort](#)
 - [Radix Sort](#)
 - [Bucket Sort](#)
- [5. Other Sorting Algorithms](#)
 - [Shell Sort — **High Interview Priority**](#)
 - [Cocktail Sort — Low Priority \(Knowledge Only\)](#)
 - [Comb Sort — Low Priority \(Knowledge Only\)](#)
 - [Cycle Sort — **High Interview Priority**](#)
 - [Pigeonhole Sort — Low Priority \(Knowledge Only\)](#)
 - [Tree Sort — Low Priority \(Knowledge Only\)](#)
 - [Tim Sort — **High Interview Priority \(conceptual\)**](#)
 - [Intro Sort — **High Interview Priority \(conceptual\)**](#)
 - [Bitonic Sort — Low Priority \(Knowledge Only\)](#)
 - [External Merge Sort — **High Interview Priority \(conceptual, for systems-design-adjacent questions\)**](#)
- [6. C++ STL Sorting](#)
 - [sort\(\)](#)
 - [stable_sort\(\)](#)
 - [partial_sort\(\)](#)
 - [nth_element\(\)](#)
 - [Summary Table](#)
- [7. Custom Comparators in C++](#)
 - [Function Comparator](#)
 - [Lambda Comparator](#)
 - [Ascending Order](#)
 - [Descending Order](#)
 - [Sorting Pairs](#)
 - [Sorting Structures](#)
 - [Multiple Conditions](#)
- [8. Sorting Different Data Types](#)
 - [Array](#)
 - [Vector](#)
 - [String](#)
 - [Pair](#)
 - [Structure](#)
 - [2D Vector](#)
 - [Intervals](#)
- [9. Sorting + DSA Patterns](#)
 - [Sorting + Two Pointers](#)

- [Sorting + Greedy](#)
- [Sorting + Intervals](#)
- [Sorting + Binary Search](#)
- [Sorting + Hashing](#)
- [10. Important Advanced Sorting Problems](#)
 - [Count Inversions](#)
 - [Kth Largest Element — Compare All Approaches](#)
 - [Kth Smallest Element](#)
 - [Quickselect \(from scratch\)](#)
- [11. Important LeetCode-Style Problems](#)
 - [1. Sort an Array \(LC 912\)](#)
 - [2-4. Bubble / Selection / Insertion Sort](#)
 - [5. Merge Sorted Array \(LC 88\)](#)
 - [6. Sort Colors \(LC 75, Dutch National Flag\)](#)
 - [7. Merge Intervals \(LC 56\)](#)
 - [8. Insert Interval \(LC 57\)](#)
 - [9. 3Sum \(LC 15\)](#)
 - [10. 4Sum \(LC 18\)](#)
 - [11. Kth Largest Element \(LC 215\)](#)
 - [12. Kth Smallest Element](#)
 - [13. Sort Characters by Frequency \(LC 451\)](#)
 - [14. Largest Number \(LC 179\)](#)
 - [15. Count Inversions](#)
 - [16. Reverse Pairs \(LC 493\)](#)
 - [17. Meeting Rooms \(LC 252\)](#)
 - [18. Minimum Platforms](#)
 - [19. Maximum Number of Events \(LC 1353\)](#)
 - [20. Merge K Sorted Lists \(LC 23\)](#)
- [12. Sorting Complexity Table](#)
- [13. How to Choose a Sorting Algorithm](#)
- [14. Common C++ Mistakes](#)
- [15. Interview Preparation](#)
 - [Beginner Questions \(15\)](#)
 - [Easy Questions \(15\)](#)
 - [Medium Questions \(20\)](#)
 - [Hard Questions \(10\)](#)
- [16. Revision Cheat Sheet \(10-15 Minute Review\)](#)
- [17. Learning Order \(and Why\)](#)

1. Sorting Basics

What is Sorting?

Sorting means arranging elements of a collection (array, vector, list) into a specific order — usually **ascending** (small to large) or **descending** (large to small).

Example:

```
Input:  [5, 2, 8, 1, 9]
Output: [1, 2, 5, 8, 9]   (ascending)
```

Why Do We Need Sorting?

- Makes **searching faster** (binary search needs sorted data — $O(\log n)$ instead of $O(n)$).
- Makes data **easier to understand and analyze** (e.g., leaderboard, exam ranks).
- Many algorithms (greedy, two-pointer, interval problems) **only work correctly on sorted data**.
- Removing duplicates, finding medians, and merging datasets become simple after sorting.

Real-World Applications

- Search engines ranking results by relevance score.

- E-commerce sites sorting products by price/rating.
- Databases using sorted indexes (B-Trees) for fast lookup.
- Contact lists sorted alphabetically.
- Scheduling problems (sort tasks by deadline/start time).
- Leaderboards, exam merit lists, file explorers (sort by name/date/size).

Comparison vs Non-Comparison Sorting

Type	How it works	Examples	Lower Bound
Comparison-based	Decides order by comparing two elements ($a < b$)	Bubble, Selection, Insertion, Merge, Quick, Heap	$O(n \log n)$
Non-comparison-based	Uses element properties (digits, range) instead of comparing pairs	Counting, Radix, Bucket	Can be $O(n)$

Comparison sorts can never beat **$O(n \log n)$** in the worst case (proven using decision trees). Non-comparison sorts can beat this because they don't rely on pairwise comparisons — but they need extra assumptions (small range, integer keys, etc).

Stable vs Unstable Sorting

A sort is **stable** if two elements with **equal keys** keep their **original relative order** after sorting.

Input: (3,'a') (1,'b') (3,'c')
 Sorted by first value:

Stable -> (1,'b') (3,'a') (3,'c') -- 'a' still comes before 'c'
 Unstable -> (1,'b') (3,'c') (3,'a') -- order among equal keys changed

Why it matters: If you first sort students by name, then stable-sort by marks, students with the same marks stay alphabetically ordered. Unstable sorts don't guarantee this.

- Stable: Bubble, Insertion, Merge, Counting, Radix, Bucket, Tim Sort
- Unstable: Selection, Quick (standard), Heap

In-Place vs Out-of-Place Sorting

- **In-place**: uses $O(1)$ or $O(\log n)$ extra memory (sorts within the original array). E.g., Bubble, Selection, Insertion, Quick, Heap.
- **Out-of-place**: needs extra memory proportional to input size. E.g., Merge Sort ($O(n)$ extra array), Counting Sort ($O(n+k)$).

Adaptive vs Non-Adaptive Sorting

- **Adaptive**: runs faster when input is already (nearly) sorted. E.g., Insertion Sort ($O(n)$ best case), Bubble Sort (with early-exit flag).
- **Non-Adaptive**: takes the same time regardless of existing order. E.g., Selection Sort, standard Merge Sort, Heap Sort.

Internal vs External Sorting

- **Internal Sorting**: entire data fits in RAM. All the algorithms in this chapter (Bubble, Merge, Quick, etc).
- **External Sorting**: data is too large for RAM (huge files, database records) — data lives on disk, and the algorithm processes it in chunks. E.g., **External Merge Sort** (covered in Section 5).

Time Complexity

Number of basic operations (comparisons/swaps) as a function of input size n . Expressed using **Big-O** notation for the worst case, and often Best/Average/Worst are all listed since they can differ a lot for sorting algorithms.

Space Complexity

Extra memory used **besides the input array**. In-place algorithms have $O(1)$ (sometimes $O(\log n)$ for recursion stack, e.g., Quick Sort).

Best, Average, and Worst Case — Why All Three Matter

- **Best Case:** the easiest input the algorithm could get (e.g., already sorted array).
- **Average Case:** expected performance over random inputs.
- **Worst Case:** hardest input (e.g., reverse-sorted array, or a bad pivot choice in Quick Sort).

Interviewers care about worst case most, because it's a guarantee. But best/average case explains *why* an algorithm like Insertion Sort is still useful in practice (nearly-sorted data is common).

```
// Example: comparing two elements is the "basic operation" we count  
if (arr[j] > arr[j + 1]) { // 1 comparison  
    swap(arr[j], arr[j + 1]); // 1 swap (3 assignments)  
}
```

Key Idea: Sorting reorders data to make later operations (search, merge, dedup) fast and simple. **When to Use:**

Whenever data needs to be ranked, searched efficiently, or fed into an algorithm that assumes order. **Interview Tip:**

Interviewers often first ask “what are the properties of this sort?” (stable? in-place? adaptive?) before asking you to code it — memorize the properties table in Section 12.

2. Basic Sorting Algorithms

Bubble Sort

Intuition

Repeatedly compare **adjacent** elements and swap them if they're in the wrong order. The largest element “bubbles up” to the end after each full pass — like bubbles rising to the surface of water.

Algorithm

1. Loop over the array `n-1` times (passes).
2. In each pass, compare every pair of adjacent elements.
3. Swap if `arr[j] > arr[j+1]`.
4. After pass `i`, the last `i` elements are guaranteed sorted (largest ones settled at the end).

Step-by-Step Working

```
Array: [5, 1, 4, 2, 8]

Pass 1:
compare(5,1) -> swap -> [1,5,4,2,8]
compare(5,4) -> swap -> [1,4,5,2,8]
compare(5,2) -> swap -> [1,4,2,5,8]
compare(5,8) -> no swap-> [1,4,2,5,8]
(8 is now in final position)

Pass 2:
compare(1,4) -> no swap
compare(4,2) -> swap -> [1,2,4,5,8]
compare(4,5) -> no swap
(5 settled)

Pass 3: no swaps needed -> array already sorted -> stop early
```

Dry Run Table

Pass	Array State	Swaps
Start	[5,1,4,2,8]	-
1	[1,4,2,5,8]	3
2	[1,2,4,5,8]	1
3	[1,2,4,5,8]	0 (sorted, break)

Optimized Bubble Sort

Add a `swapped` flag. If no swaps happen in a pass, the array is already sorted — stop early. This makes best case $O(n)$ instead of $O(n^2)$.

C++ Implementation

```
void bubbleSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        if (!swapped)
            break;
    }
}
```

Line-by-Line Explanation

- `int n = arr.size();` — store array length.
- `for (int i = 0; i < n - 1; i++)` — outer loop controls number of passes. After pass `i`, the last `i` elements are sorted, so we only need `n-1` passes max.
- `bool swapped = false;` — tracks whether any swap happened this pass (optimization).
- `for (int j = 0; j < n - i - 1; j++)` — inner loop compares adjacent pairs. `n - i - 1` shrinks each pass since the tail is already sorted.
- `if (arr[j] > arr[j + 1])` — compare adjacent elements.
- `swap(arr[j], arr[j + 1]);` — swap if out of order.
- `swapped = true;` — mark that this pass did work.
- `if (!swapped) break;` — if a full pass had zero swaps, array is sorted; exit early.

Complexity

Case	Time
Best (already sorted)	$O(n)$
Average	$O(n^2)$
Worst (reverse sorted)	$O(n^2)$
Space	$O(1)$

Stable/In-Place Properties

- **Stable:** Yes (equal elements never swap, since we only swap on strict `>`).
- **In-place:** Yes ($O(1)$ extra space).
- **Adaptive:** Yes (with the `swapped` optimization).

Advantages / Disadvantages

- ✓ Extremely simple to understand and implement.
- ✓ Stable, in-place, adaptive.
- ✗ $O(n^2)$ worst/average — impractical for large datasets.
- ✗ Rarely used in production; mainly educational.

Interview Questions

1. Why is Bubble Sort stable? (Because we only swap strictly-greater adjacent pairs — equal elements never swap.)
2. How do you make Bubble Sort adaptive? (Add the `swapped` flag.)
3. Modify Bubble Sort to sort in descending order. (Flip the comparison to `arr[j] < arr[j+1]`.)

Key Idea: Repeatedly swap adjacent out-of-order pairs so the largest “bubbles” to the end each pass. **When to Use:** Teaching purposes, tiny arrays, or when you need a dead-simple stable sort and performance doesn’t matter. **Complexity:** Best $O(n)$, Average/Worst $O(n^2)$, Space $O(1)$. **Common Mistakes:** Forgetting `n - i - 1` (causes out-of-bounds or redundant comparisons); forgetting the early-exit optimization. **Interview Tip:** Always mention the `swapped` optimization — it shows you understand adaptive behavior, not just rote memorization.

Selection Sort

Intuition

Find the **minimum** element in the unsorted part of the array and swap it to the front. Repeat for the remaining unsorted part. Unlike Bubble Sort, Selection Sort does at most `n-1` swaps total (good when swaps are expensive, e.g., writing to flash memory).

Finding Minimum

For each position `i`, scan the rest of the array (`i+1` to `n-1`) to find the index of the smallest element, then swap it into position `i`.

Swapping

Only **one swap per outer iteration** (not per comparison, unlike Bubble Sort).

Dry Run

Array: [29, 10, 14, 37, 13]

i=0: min in [29,10,14,37,13] is 10 at idx1 -> swap(0,1) -> [10,29,14,37,13]
i=1: min in [29,14,37,13] is 13 at idx4 -> swap(1,4) -> [10,13,14,37,29]
i=2: min in [14,37,29] is 14 at idx2 -> no swap -> [10,13,14,37,29]
i=3: min in [37,29] is 29 at idx4 -> swap(3,4) -> [10,13,14,29,37]

Result: [10,13,14,29,37]

C++ Code

```
void selectionSort(vector<int>& arr) {  
    int n = arr.size();  
  
    for (int i = 0; i < n - 1; i++) {  
        int minIdx = i;  
  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIdx]) {  
                minIdx = j;  
            }  
        }  
  
        if (minIdx != i) {  
            swap(arr[i], arr[minIdx]);  
        }  
    }  
}
```

`minIdx` tracks the index of the smallest element found so far in the unsorted suffix. After the inner loop finishes scanning, we swap it into place — exactly one swap per outer iteration.

Complexity

Case	Time
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$
Space	$O(1)$

Note: Selection Sort has **no best case improvement** — it always scans the full remaining array even if already sorted. This makes it **non-adaptive**.

Stable/In-Place

- **Stable:** No, by default (swapping can jump an equal element out of order). Example: [4a, 4b, 1] → swap min(1) with index 0 → [1, 4b, 4a] — order of the two 4's flipped.
- **In-place:** Yes.

Common Mistakes

- Swapping inside the inner loop instead of after finding the true minimum (turns it into a different, less efficient algorithm).
- Forgetting the `if (minIdx != i)` check (not wrong, just an unnecessary self-swap — harmless but wasteful).
- Assuming it's stable (it isn't, by default).

Key Idea: Repeatedly select the minimum of the unsorted part and place it at the front. **When to Use:** When the number of swaps must be minimized (e.g., writes are costly), and array is small. **Complexity:** $O(n^2)$ always, $O(1)$ space. **Common Mistakes:** Assuming stability; swapping too early. **Interview Tip:** Mention it does exactly $n-1$ swaps max — useful when memory writes are expensive (e.g., EEPROM).

Insertion Sort

Intuition

Think of sorting playing cards in your hand: pick the next card and insert it into the correct position among the already-sorted cards to its left.

Why It Is Useful for Nearly Sorted Arrays

If the array is already sorted (or almost), each element only needs to shift 0 or 1 positions → **$O(n)$ best case**. This makes Insertion Sort excellent for nearly-sorted data or small arrays (many library `sort()` implementations switch to insertion sort for small sub-arrays).

key, and Shifting Elements

- `key` = the current element being inserted into the sorted left portion.
- We shift elements greater than `key` one position right to make room, then place `key` in the gap.

Dry Run

```
Array: [12, 11, 13, 5, 6]

i=1: key=11. Compare with 12 -> shift 12 right -> [12,12,13,5,6] -> place 11 -> [11,12,13,5,6]
i=2: key=13. 13>12, no shift -> [11,12,13,5,6]
i=3: key=5. Shift 13,12,11 right -> place 5 at start -> [5,11,12,13,6]
i=4: key=6. Shift 13,12,11 right (11>6 shift, but 5<6 stop) -> [5,6,11,12,13]

Result: [5,6,11,12,13]
```

C++ Implementation

```
void insertionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // shift right
            j--;
        }

        arr[j + 1] = key; // insert key in the gap
    }
}
```

- `key = arr[i]` — the element we want to place correctly.
- `j = i - 1` — start comparing from the element just left of `key`.
- `while (j >= 0 && arr[j] > key)` — shift every element greater than `key` one step right.
- `arr[j + 1] = key;` — after the loop, `j+1` is the correct gap for `key`.

Complexity

Case	Time	Reason
Best	$O(n)$	Already sorted — inner while never runs
Average	$O(n^2)$	
Worst	$O(n^2)$	Reverse sorted — max shifting
Space	$O(1)$	

Common Interview Questions

1. Why is Insertion Sort preferred for small/nearly-sorted arrays? (Best case $O(n)$, low constant factor, cache-friendly.)
2. How is it used inside hybrid sorts like Tim Sort / Intro Sort? (As the base case for small sub-arrays, typically size $< 16-32$.)
3. Convert Insertion Sort to sort a linked list. (Same idea, but insert via pointer manipulation instead of index shifting.)

Key Idea: Grow a sorted prefix one element at a time by inserting each new element into its correct position. **When to Use:** Small arrays, nearly-sorted data, online sorting (data arriving one at a time), and as a sub-routine in hybrid sorts (Tim Sort, Intro Sort). **Complexity:** Best $O(n)$, Average/Worst $O(n^2)$, Space $O(1)$. **Common Mistakes:** Off-by-one on `j` bounds; forgetting `j >= 0` check causing out-of-bounds access. **Interview Tip:** Insertion Sort is **stable** and **adaptive** — highlight both when comparing it to Selection Sort.

3. Important Efficient Sorting Algorithms

Merge Sort

Divide and Conquer

Merge Sort splits the array into halves recursively (**divide**) until each piece has 1 element (trivially sorted), then merges sorted halves back together (**conquer/combine**).

Divide Phase

```
[38, 27, 43, 3, 9, 82, 10]
  /      \
[38,27,43,3]  [9,82,10]
 /  \      /  \
[38,27] [43,3] [9,82] [10]
 /  \      /  \      /  \
[38][27] [43][3] [9][82]
```

Keep splitting until sub-arrays have size 1 (base case).

Merge Phase

Merge sorted pairs back up, combining two sorted arrays into one sorted array:

```
[38]+[27] -> [27,38]
[43]+[3]  -> [3,43]
[27,38]+[3,43] -> [3,27,38,43]
[9]+[82] -> [9,82]
[9,82]+[10] -> [9,10,82]
[3,27,38,43]+[9,10,82] -> [3,9,10,27,38,43,82]
```

Recursion / Base Case

- **Base case:** sub-array of size ≤ 1 is already sorted — return.
- **Recursive case:** split into `low..mid` and `mid+1..high`, sort each half, then merge.

Recursive Tree

Depth of the tree is $\log n$ (halving each time). At each level, merging touches all n elements $\rightarrow$ total work = $O(n \log n)$.

Dry Run (small example)

```
Array: [5, 2, 4, 1]

split -> [5,2] [4,1]
split -> [5][2] [4][1]
merge [5]+[2] -> [2,5]
merge [4]+[1] -> [1,4]
merge [2,5]+[1,4]:
  compare 2 vs 1 -> take 1 -> [1]
  compare 2 vs 4 -> take 2 -> [1,2]
  compare 5 vs 4 -> take 4 -> [1,2,4]
  remaining 5 -> [1,2,4,5]

Result: [1,2,4,5]
```

C++ Implementation

```
void merge(vector<int>& arr, int low, int mid, int high) {
    vector<int> temp; // extra array
    int left = low, right = mid + 1;

    while (left <= mid && right <= high) {
        if (arr[left] <= arr[right]) {
            temp.push_back(arr[left]);
            left++;
        } else {
            temp.push_back(arr[right]);
            right++;
        }
    }

    while (left <= mid) temp.push_back(arr[left]);
    while (right <= high) temp.push_back(arr[right]);

    for (int i = low; i <= high; i++) arr[i] = temp[i - low];
}
```

```

        right++;
    }
}

while (left <= mid) {           // leftover left half
    temp.push_back(arr[left]);
    left++;
}
while (right <= high) {        // leftover right half
    temp.push_back(arr[right]);
    right++;
}

for (int i = low; i <= high; i++) {
    arr[i] = temp[i - low];    // copy back
}
}

void mergeSort(vector<int>& arr, int low, int high) {
    if (low >= high) return;    // base case: 0 or 1 element

    int mid = low + (high - low) / 2;
    mergeSort(arr, low, mid);   // sort left half
    mergeSort(arr, mid + 1, high); // sort right half
    merge(arr, low, mid, high); // merge sorted halves
}

```

class Solution version (LeetCode style)

```

class Solution {
public:
    void merge(vector<int>& arr, int low, int mid, int high) {
        vector<int> temp;
        int left = low, right = mid + 1;
        while (left <= mid && right <= high) {
            if (arr[left] <= arr[right]) temp.push_back(arr[left++]);
            else temp.push_back(arr[right++]);
        }
        while (left <= mid) temp.push_back(arr[left++]);
        while (right <= high) temp.push_back(arr[right++]);
        for (int i = low; i <= high; i++) arr[i] = temp[i - low];
    }

    void mergeSort(vector<int>& arr, int low, int high) {
        if (low >= high) return;
        int mid = low + (high - low) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }

    vector<int> sortArray(vector<int>& nums) {
        mergeSort(nums, 0, nums.size() - 1);
        return nums;
    }
};

```

Why Extra Memory Is Required

Merging two sorted halves in-place efficiently isn't possible without shifting elements (which costs extra time). Using a `temp` array lets us merge in $O(n)$ time at the cost of $O(n)$ extra space.

Time Complexity Derivation

Recurrence: $T(n) = 2T(n/2) + O(n)$ (two recursive calls on half input + $O(n)$ merge work). By the **Master Theorem** ($a=2$, $b=2$, $f(n)=O(n)$): since $f(n) = \Theta(n^{\log_b a}) = \Theta(n^1)$, this falls in **Case 2** $\rightarrow T(n) = O(n \log n)$.

This holds for **best, average, and worst case** — Merge Sort's runtime doesn't depend on input order.

Space Complexity

$O(n)$ for the temporary merge array + $O(\log n)$ recursion stack = **$O(n)$** overall.

Key Idea: Divide array into halves, sort each half recursively, then merge sorted halves. **When to Use:** When you need guaranteed $O(n \log n)$ and stability (e.g., sorting linked lists, external sorting, counting inversions). **Complexity:** $O(n \log n)$ all cases, $O(n)$ space. **Common Mistakes:** Off-by-one in `mid / high` bounds; forgetting to copy `temp` back into `arr`; integer

overflow with $(low+high)/2$ (use $low + (high-low)/2$). **Interview Tip:** Merge Sort is the go-to answer whenever a problem needs stable $O(n \log n)$ sorting or when the “merge step” itself can be repurposed (e.g., counting inversions, merging k lists).

Quick Sort

Divide and Conquer

Pick a **pivot**, partition the array so elements smaller than pivot go left and larger go right, then recursively sort both sides. Unlike Merge Sort, the “work” happens during **divide** (partition), not during combine (nothing to combine — array is sorted in place).

Pivot

Any chosen element to partition around — commonly the last element, first element, middle, or a random element.

Partition

Rearranges the array so: `[elements < pivot] pivot [elements >= pivot]`, and returns the pivot’s final sorted index.

Lomuto Partition (pivot = last element)

```
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high]; // choose last element as pivot
    int i = low - 1;        // boundary of "smaller than pivot" region

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]); // place pivot in correct position
    return i + 1;                // pivot's final index
}
```

Line-by-line: - `pivot = arr[high]` — fix the pivot value. - `i = low - 1` — marks the last index known to be `< pivot`. - Loop `j` scans the array; whenever `arr[j] < pivot`, we expand the “small” region by incrementing `i` and swapping `arr[i]` with `arr[j]`. - After the loop, everything in `[low, i]` is `< pivot`. - `swap(arr[i+1], arr[high])` places the pivot right after the “small” region — its correct sorted position. - Return `i + 1`, the pivot’s index, so the caller knows where to split for recursion.

Hoare Partition (two pointers from both ends)

```
int hoarePartition(vector<int>& arr, int low, int high) {
    int pivot = arr[low];
    int i = low - 1, j = high + 1;

    while (true) {
        do { i++; } while (arr[i] < pivot);
        do { j--; } while (arr[j] > pivot);

        if (i >= j) return j;
        swap(arr[i], arr[j]);
    }
}
```

Hoare partition does fewer swaps on average and is generally faster, but the pivot doesn’t land in its final sorted position (recursion bounds differ: `[low, j]` and `[j+1, high]`).

Recursive Calls

```
void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // left of pivot
        quickSort(arr, pi + 1, high); // right of pivot
    }
}
```

```
}
```

Dry Run (Lomuto, pivot = last element)

Array: [10, 80, 30, 90, 40], low=0, high=4, pivot=40

```
i=-1
j=0: arr[0]=10 < 40 -> i=0, swap(0,0) -> [10,80,30,90,40]
j=1: arr[1]=80 not < 40 -> skip
j=2: arr[2]=30 < 40 -> i=1, swap(1,2) -> [10,30,80,90,40]
j=3: arr[3]=90 not < 40 -> skip

swap(i+1=2, high=4) -> [10,30,40,90,80]
pivot index = 2 -> recurse on [10,30] and [90,80]
```

Best / Average / Worst Case

Case	When it happens	Time
Best	Pivot always splits array in half	$O(n \log n)$
Average	Random-ish splits	$O(n \log n)$
Worst	Pivot is always smallest/largest (sorted/reverse-sorted array with naive pivot)	$O(n^2)$

Why Worst Case Becomes $O(n^2)$

If the pivot is always the smallest or largest element, one partition has 0 elements and the other has $n-1$. Recurrence becomes $T(n) = T(n-1) + O(n)$, which sums to $O(n^2)$ — essentially degrades to Selection-Sort-like behavior.

```
Sorted array [1,2,3,4,5], pivot=last element(5) every time:
Partition puts everything on one side -> unbalanced splits ->  $O(n^2)$ 
```

Randomized Quick Sort

Pick a **random index** as pivot (swap it with the last element, then run normal Lomuto partition). This makes the worst case extremely unlikely for any *fixed* input, giving expected $O(n \log n)$.

```
int randomPartition(vector<int>& arr, int low, int high) {
    int randIdx = low + rand() % (high - low + 1);
    swap(arr[randIdx], arr[high]);
    return partition(arr, low, high); // normal Lomuto partition
}
```

Quick Sort vs Merge Sort

Aspect	Quick Sort	Merge Sort
Worst case	$O(n^2)$	$O(n \log n)$
Space	$O(\log n)$ (in-place)	$O(n)$
Stable	No	Yes
Cache performance	Better (in-place)	Worse (extra array)
Typical use	General-purpose in-memory sort (e.g., <code>std::sort</code>)	Linked lists, external sort, when stability needed

Key Idea: Partition around a pivot so smaller elements go left, larger go right, then recurse on both sides. **When to Use:** General-purpose fast in-memory sorting when stability isn't required (this is what `std::sort` uses, combined with heap sort as a worst-case fallback — Intro Sort). **Complexity:** Best/Average $O(n \log n)$, Worst $O(n^2)$, Space $O(\log n)$. **Common Mistakes:** Wrong partition boundary recursion (off-by-one causes infinite recursion or missed elements); not randomizing pivot on sorted input; using Hoare's returned index incorrectly for recursion bounds (Hoare requires `[low, j]` and `[j+1, high]`, NOT `pivotIndex-1 / pivotIndex+1`). **Interview Tip:** Always mention **Randomized Quick Sort** to show you know how to avoid the $O(n^2)$ worst case in practice.

Heap Sort

What Is a Heap?

A **heap** is a **complete binary tree** stored in an array, satisfying the heap property.

Complete Binary Tree

Every level is fully filled except possibly the last, which is filled left to right. This lets us store the tree in a plain array with no gaps.

Max Heap / Min Heap

- **Max Heap**: every parent $\geq$ its children (root = maximum element).
- **Min Heap**: every parent $\leq$ its children (root = minimum element).

Array Representation of Heap

Max Heap: Array: [90, 80, 70, 40, 30, 60, 10]

```
      90
     /  \
    80   70
   / \  / \
  40 30 60 10
```

Parent/Child Formulas (0-indexed array)

```
parent(i) = (i - 1) / 2
left(i)   = 2*i + 1
right(i)  = 2*i + 2
```

Heapify

`heapify(arr, n, i)` assumes the subtrees rooted at `left(i)` and `right(i)` are already valid max-heaps, and fixes the heap property at node `i` by “sinking” it down if needed.

```
void heapify(vector<int>& arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest); // recurse down
    }
}
```

Building a Heap

Call `heapify` on every non-leaf node, starting from the last non-leaf ($n/2 - 1$) up to the root. This is $O(n)$, not $O(n \log n)$, due to a tighter mathematical bound (most nodes are near the bottom and need little work).

```
void buildMaxHeap(vector<int>& arr, int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}
```

Heap Sort

1. Build a max heap from the array.
2. Swap root (max element) with the last element — max is now in its final sorted position.
3. Reduce heap size by 1, `heapify` the root to restore heap property.
4. Repeat until heap size is 1.

```

void heapSort(vector<int>& arr) {
    int n = arr.size();

    buildMaxHeap(arr, n);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);    // move current max to the end
        heapify(arr, i, 0);     // heapify reduced heap (size i)
    }
}

```

Dry Run

Array: [4, 10, 3, 5, 1]

Build max heap -> [10, 5, 3, 4, 1]

i=4: swap(0,4) -> [1,5,3,4,10], heapify(size=4,root=0) -> [5,4,3,1,10]

i=3: swap(0,3) -> [1,4,3,5,10], heapify(size=3,root=0) -> [4,1,3,5,10]

i=2: swap(0,2) -> [3,1,4,5,10], heapify(size=2,root=0) -> [3,1,4,5,10]

i=1: swap(0,1) -> [1,3,4,5,10]

Result: [1,3,4,5,10]

Complexity

Operation	Time
Build heap	$O(n)$
Extract max + heapify ($\times n$)	$O(n \log n)$
Total	$O(n \log n)$ (best, average, worst — always)
Space	$O(1)$ (in-place, $O(\log n)$ recursion stack for heapify)

Stable? No (swaps can reorder equal elements). **In-place?** Yes.

Key Idea: Build a max heap, repeatedly move the max to the end and shrink the heap. **When to Use:** When you need guaranteed $O(n \log n)$ with $O(1)$ extra space (Merge Sort needs $O(n)$ space) — used as the worst-case fallback inside Intro Sort. **Complexity:** $O(n \log n)$ all cases, $O(1)$ space. **Common Mistakes:** Confusing 0-indexed and 1-indexed parent/child formulas; forgetting to shrink n when re-heapifying after each extraction; calling `heapify` on leaf nodes unnecessarily (wasteful but not wrong). **Interview Tip:** Know that `buildMaxHeap` is $O(n)$ (not $O(n \log n)$) — a very common interview trick question about complexity analysis.

4. Non-Comparison Sorting

Counting Sort

Frequency Array

Count how many times each value appears, using an auxiliary “count” array indexed by value.

Finding Min/Max

Find `minVal` and `maxVal` in the array first, so the count array only needs size `(maxVal - minVal + 1)` instead of assuming values start at 0.

Handling Negative Numbers

Shift every value by `-minVal` when indexing into the count array (so the smallest value maps to index 0), and shift back when reading results out.

Dry Run

```
Array: [4, 2, 2, 8, 3, 3, 1]
min=1, max=8 -> range = 8

count[]: idx(value-1): 1->1, 2->2, 3->2, 4->1, 8->1
count = [1,2,2,1,0,0,0,1] (indices represent values 1..8)

prefix sum -> [1,3,5,6,6,6,7] (tells final position of each value)

Place elements from the back (for stability):
1 (freq at value1=1) -> position 1 -> output[0]=1
...
Final output: [1,2,2,3,3,4,8]
```

C++ Implementation (stable version using prefix sums)

```
void countingSort(vector<int>& arr) {
    if (arr.empty()) return;

    int maxVal = *max_element(arr.begin(), arr.end());
    int minVal = *min_element(arr.begin(), arr.end());
    int range = maxVal - minVal + 1;

    vector<int> count(range, 0);
    for (int x : arr) count[x - minVal]++;           // frequency

    for (int i = 1; i < range; i++)
        count[i] += count[i - 1];                   // prefix sum -> final positions

    vector<int> output(arr.size());
    for (int i = arr.size() - 1; i >= 0; i--) {      // iterate backward for stability
        output[count[arr[i] - minVal] - 1] = arr[i];
        count[arr[i] - minVal]--;
    }

    arr = output;
}
```

Complexity

- Time: $O(n + k)$ where k = range of values (`maxVal - minVal + 1`).
- Space: $O(n + k)$.
- Stable: Yes (with the backward-iteration + prefix-sum technique).

Limitations

- Only works well when k (range of values) is not much larger than n — if values range from 1 to 10^9 , the count array would be huge.
- Only works on integers (or things easily mapped to small integer keys).
- Not comparison-based, so it beats the $O(n \log n)$ lower bound — but only under these constraints.

Radix Sort

Digit-by-Digit Sorting

Sort numbers digit by digit, starting from the **least significant digit (LSD)** to the most significant, using a **stable** sort (Counting Sort) at each digit position.

LSD Radix Sort

Array: [170, 45, 75, 90, 802, 24, 2, 66]

Sort by 1's digit (stable counting sort):
[170, 90, 802, 2, 24, 45, 75, 66]

Sort by 10's digit:
[802, 2, 24, 45, 66, 170, 75, 90]

Sort by 100's digit:
[2, 24, 45, 66, 75, 90, 170, 802]

Result: sorted!

Counting Sort Inside Radix Sort

Each pass uses counting sort on the digit at the current place value (1s, 10s, 100s, ...), which is always in range [0, 9] — so $k = 10$, making each pass $O(n)$.

C++ Implementation

```
void countingSortByDigit(vector<int>& arr, int exp) {
    int n = arr.size();
    vector<int> output(n);
    vector<int> count(10, 0);

    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) {           // backward for stability
        int digit = (arr[i] / exp) % 10;
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }

    arr = output;
}

void radixSort(vector<int>& arr) {
    if (arr.empty()) return;
    int maxVal = *max_element(arr.begin(), arr.end());

    for (int exp = 1; maxVal / exp > 0; exp *= 10) {
        countingSortByDigit(arr, exp);
    }
}
```

Complexity

- Time: $O(d \cdot (n + k))$ where d = number of digits in the max number, $k = 10$ (base 10). Effectively $O(d \cdot n)$.
- Space: $O(n + k)$.
- Stable: Yes.
- Only works on non-negative integers directly (needs modification for negatives — split into negative/positive groups and sort separately).

Bucket Sort

Buckets

Divide the range of input values into several “buckets”, and distribute elements into their appropriate bucket.

Distribution

Each element x (assuming values in $[0, 1)$) goes into bucket $\text{floor}(x * \text{numBuckets})$.

Sorting Buckets

Sort each bucket individually (usually with Insertion Sort, since buckets are small).

Combining Buckets

Concatenate all buckets in order to get the final sorted array.

C++ Implementation (for floats in $[0, 1)$)

```
void bucketSort(vector<float>& arr) {
    int n = arr.size();
    vector<vector<float>> buckets(n);

    for (float x : arr) {
        int idx = n * x;           // assumes 0 <= x < 1
        buckets[idx].push_back(x);
    }

    for (auto& bucket : buckets) {
        sort(bucket.begin(), bucket.end()); // insertion sort for small buckets in practice
    }

    int index = 0;
    for (auto& bucket : buckets) {
        for (float x : bucket) {
            arr[index++] = x;
        }
    }
}
```

Complexity

- Average: $O(n + k)$ when data is uniformly distributed across buckets.
- Worst: $O(n^2)$ if all elements land in one bucket.
- Space: $O(n + k)$.
- Stable: Yes, if the inner sort is stable.

Use Cases

- Data uniformly distributed over a known range (e.g., floating-point values in $[0,1)$).
- External sorting when data can be bucketed by range and each bucket fits in memory.

Key Idea (all three): Skip pairwise comparisons; use the *value* itself (digit, frequency, range) to directly determine position. **When to Use:** Integers with small range (Counting), integers/strings with fixed digit count (Radix), uniformly distributed floats (Bucket). **Complexity:** Counting $O(n+k)$, Radix $O(d \cdot n)$, Bucket $O(n+k)$ average / $O(n^2)$ worst. **Common Mistakes:** Forgetting these need extra assumptions about data (range, distribution) — they are NOT general-purpose replacements for comparison sorts. **Interview Tip:** If asked “can you sort faster than $O(n \log n)$?”, the answer is yes — *if* you know something extra about the data (small integer range, fixed digits, uniform distribution). Always state the assumption.

5. Other Sorting Algorithms

Shell Sort — High Interview Priority

An extension of Insertion Sort that first sorts elements far apart (using a “gap” sequence), progressively shrinking the gap to 1 (a final normal insertion sort). This moves out-of-place elements faster than plain insertion sort.

```
void shellSort(vector<int>& arr) {
    int n = arr.size();
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}
```

Complexity: depends on gap sequence, typically $O(n^{1.3})$ to $O(n^2)$ worst case; $O(1)$ space; **not stable**.

Cocktail Sort — Low Priority (Knowledge Only)

Bidirectional Bubble Sort: passes go left-to-right then right-to-left alternately. Same $O(n^2)$ complexity as Bubble Sort but slightly fewer passes for certain patterns (“turtles” near the wrong end move faster).

Comb Sort — Low Priority (Knowledge Only)

Like Bubble Sort but starts comparing elements with a large gap (shrinking by factor ~ 1.3 each pass) instead of $\text{gap}=1$, reducing “turtle” elements that make Bubble Sort slow. Average complexity better than $O(n^2)$ in practice, but not commonly asked.

Cycle Sort — High Interview Priority

Minimizes the number of writes to the array (useful when writes are very expensive). Places each element directly into its correct final position by computing how many elements are smaller than it. Frequently appears in problems like “find missing/duplicate number” via **index-based placement**.

```
void cycleSort(vector<int>& arr) {
    int n = arr.size();
    for (int cycleStart = 0; cycleStart < n - 1; cycleStart++) {
        int item = arr[cycleStart];
        int pos = cycleStart;
        for (int i = cycleStart + 1; i < n; i++)
            if (arr[i] < item) pos++;
        if (pos == cycleStart) continue;
        while (item == arr[pos]) pos++;
        swap(arr[pos], item);
        while (pos != cycleStart) {
            pos = cycleStart;
            for (int i = cycleStart + 1; i < n; i++)
                if (arr[i] < item) pos++;
            while (item == arr[pos]) pos++;
            swap(arr[pos], item);
        }
    }
}
```

Complexity: $O(n^2)$ time, $O(1)$ space, **minimizes writes** to exactly n (theoretically minimal).

Pigeonhole Sort — Low Priority (Knowledge Only)

Similar to Counting Sort — create a “pigeonhole” (bucket) for every possible value in the range, place elements in their

hole, then read them back in order. Requires range $\approx n$ for efficiency.

Tree Sort — Low Priority (Knowledge Only)

Insert all elements into a Binary Search Tree, then do an in-order traversal to get sorted output. $O(n \log n)$ average (if tree stays balanced, e.g., using a self-balancing BST), $O(n^2)$ worst case (unbalanced BST from sorted input).

Tim Sort — High Interview Priority (conceptual)

Hybrid of **Merge Sort + Insertion Sort**, used by Python's `sort()` / `sorted()` and Java's `Collections.sort()` for objects. Splits the array into small “runs”, sorts each run with Insertion Sort, then merges runs using Merge Sort logic. Exploits existing order in real-world data. Time: $O(n \log n)$ worst, **$O(n)$ best** (already sorted). Stable.

Intro Sort — High Interview Priority (conceptual)

Hybrid of **Quick Sort + Heap Sort + Insertion Sort**, used by C++ STL's `std::sort()`. Starts with Quick Sort; if recursion depth gets too large (indicating a bad pivot pattern, risking $O(n^2)$), it switches to Heap Sort to guarantee $O(n \log n)$; switches to Insertion Sort for small sub-arrays. Gives the speed of Quick Sort with the worst-case guarantee of Heap Sort.

Bitonic Sort — Low Priority (Knowledge Only)

A parallel sorting algorithm (works on GPUs) — sorts a “bitonic sequence” (first increasing, then decreasing) using a network of comparators. $O(\log^2 n)$ time on parallel hardware, $O(n \log^2 n)$ sequential. Rarely asked outside of parallel computing contexts.

External Merge Sort — High Interview Priority (conceptual, for systems-design-adjacent questions)

Used when data doesn't fit in RAM (huge files/databases): 1. Split data into chunks that fit in memory, sort each chunk (in-memory sort), write sorted chunks to disk. 2. Merge sorted chunks using a k-way merge (min-heap of size k) reading small buffers from each chunk. This is how databases sort huge tables and how `sort` (Unix command) works on huge files.

Interview Tip: You don't need to hand-code Tim Sort or Intro Sort in an interview — but you **MUST** be able to explain *why* they're hybrids and what problem each hybrid choice solves (Tim Sort exploits real-world “runs”; Intro Sort avoids Quick Sort's $O(n^2)$ worst case).

6. C++ STL Sorting

sort()

```
sort(arr.begin(), arr.end()); // ascending
sort(arr.begin(), arr.end(), greater<int>()); // descending
```

- Works on any random-access container (vector, array, deque).
- Implementation: **Intro Sort** (Quick Sort + Heap Sort + Insertion Sort hybrid).
- Time: **$O(n \log n)$** average and worst case (thanks to the Heap Sort fallback).
- Space: $O(\log n)$.
- **Not stable** (equal elements may be reordered).

stable_sort()

```
stable_sort(arr.begin(), arr.end());
```

- Guarantees equal elements keep their relative order.
- Implementation: typically **Merge Sort** (or a hybrid with Insertion Sort for small ranges).
- Time: $O(n \log n)$ — but $O(n \log^2 n)$ if extra memory isn't available (falls back to in-place merge).
- Space: $O(n)$ (needs a temp buffer).
- **Use when:** sorting objects by one key while preserving original order among ties (e.g., sort employees by department, keep name order intact).

partial_sort()

```
partial_sort(arr.begin(), arr.begin() + k, arr.end());
```

- Sorts only the smallest `k` elements into their correct sorted positions at the front; the rest of the array is left in unspecified order.
- Time: **$O(n \log k)$** — much faster than fully sorting when $k \ll n$.
- **Use when:** you need the “top k smallest/largest” elements, in order (e.g., top 3 leaderboard scores).

nth_element()

```
nth_element(arr.begin(), arr.begin() + k, arr.end());
```

- Rearranges the array so the element at index `k` is the one that *would* be there if the array were fully sorted. All elements before it are $\leq$ it, all after are $\geq$ it — but neither side is sorted internally.
- Time: **$O(n)$** average (based on Quickselect).
- **Use when:** you only need the k-th smallest/largest element (e.g., median, Kth largest) without needing full order — much faster than sorting.

Summary Table

Function	Guarantees	Time	Use Case
<code>sort()</code>	Full order, not stable	$O(n \log n)$	General sorting
<code>stable_sort()</code>	Full order, stable	$O(n \log n)$	Sort while preserving ties' order
<code>partial_sort()</code>	First k sorted	$O(n \log k)$	Top-k elements, in order
<code>nth_element()</code>	k-th element correct, unsorted rest	$O(n)$ avg	Just need median / Kth element

Interview Tip: If asked “how do you find the Kth largest element efficiently in C++ using STL alone”, the answer is `nth_element` — $O(n)$ average vs $O(n \log n)$ for a full sort.

7. Custom Comparators in C++

A comparator tells `sort()` **how to decide order** between two elements. It must return `true` if the first argument should come **before** the second.

Function Comparator

```
bool compare(pair<int,int>& a, pair<int,int>& b) {  
    return a.second < b.second; // sort by second value ascending  
}  
  
sort(v.begin(), v.end(), compare);
```

Lambda Comparator

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {  
    return a.second < b.second;  
});
```

`[](auto &a, auto &b) { ... }` is an anonymous inline function. `auto &` avoids copying elements — important for structs/pairs. The body is exactly the same logic as the named function comparator above.

How the Comparator Works Internally

`sort()` repeatedly asks the comparator “should `a` come before `b`?” during its internal Intro Sort logic. If `compare(a, b)` returns `true`, `a` is placed before `b`. The comparator must define a **strict weak ordering** — never return `true` for `compare(a,a)`, and must be consistent (if `compare(a,b)` and `compare(b,c)` are both true, `compare(a,c)` must also be true). Violating this causes **undefined behavior** (crashes, infinite loops).

Ascending Order

```
sort(v.begin(), v.end()); // default, uses operator<  
sort(v.begin(), v.end(), [](int a, int b){ return a < b; });
```

Descending Order

```
sort(v.begin(), v.end(), greater<int>());  
sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
```

Sorting Pairs

```
vector<pair<int,int>> v = {{3,1},{1,2},{2,0}};  
sort(v.begin(), v.end()); // default: sorts by first, then by second (lexicographic)  
// Result: {1,2},{2,0},{3,1}
```

Sorting Structures

```
struct Student {  
    string name;  
    int marks;  
};  
  
vector<Student> students;  
sort(students.begin(), students.end(), [](const Student& a, const Student& b) {  
    return a.marks > b.marks; // descending by marks  
});
```

Multiple Conditions

Sort by first field

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {  
    return a.first < b.first;  
});
```

Sort by second field

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {  
    return a.second < b.second;  
});
```

If first values are equal, sort by second value

```
sort(v.begin(), v.end(), [](auto &a, auto &b) {  
    if (a.first != b.first)  
        return a.first < b.first;    // primary key  
    return a.second < b.second;    // tie-breaker  
});
```

This pattern (primary key, then tie-breaker) generalizes to any number of fields:

```
sort(students.begin(), students.end(), [](const Student& a, const Student& b) {  
    if (a.marks != b.marks) return a.marks > b.marks;    // higher marks first  
    return a.name < b.name;    // tie -> alphabetical  
});
```

Key Idea: A comparator is a “should `a` come before `b`?” function — chain conditions for multi-field sorting. **When to Use:** Any time you sort structs, pairs, or need non-default order. **Common Mistakes:** Using `<=` instead of `<` in a comparator (breaks strict weak ordering, causes crashes); forgetting the tie-breaker (unstable results for equal primary keys) — use `stable_sort` if tie order from input must be preserved. **Interview Tip:** Always say the phrase “strict weak ordering” when discussing comparators — it signals you understand *why* `<=` comparators crash `std::sort`.

8. Sorting Different Data Types

Array

```
int arr[] = {5, 2, 8, 1};
sort(arr, arr + 4); // pass raw pointers: begin, end
// arr -> {1,2,5,8}
```

Vector

```
vector<int> arr = {5, 2, 8, 1};
sort(arr.begin(), arr.end());
// arr -> {1,2,5,8}
```

String

```
string s = "dcba";
sort(s.begin(), s.end());
// s -> "abcd" (strings support iterators just like vectors of char)
```

Pair

```
vector<pair<int,int>> v = {{2,3},{1,5},{1,2}};
sort(v.begin(), v.end());
// default lexicographic sort: {1,2},{1,5},{2,3}
```

Structure

```
struct Student { string name; int marks; };
vector<Student> v = { {"Bob",80}, {"Amy",95} };

sort(v.begin(), v.end(), [](const Student& a, const Student& b) {
    return a.marks > b.marks; // structs need an explicit comparator (no default <)
});
```

2D Vector

```
vector<vector<int>> v = {{3,1},{1,5},{2,0}};

sort(v.begin(), v.end()); // default: lexicographic comparison of inner vectors
// {1,5},{2,0},{3,1}

// Sort by second column instead:
sort(v.begin(), v.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[1] < b[1];
});
```

Intervals

```
vector<vector<int>> intervals = {{1,3},{2,6},{8,10},{15,18}};

// Sort by start time (the standard first step for interval problems)
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[0] < b[0];
});
```

Practical Note: For `vector<int>` and `pair<int,int>`, C++ already provides a default `operator<` (lexicographic), so `sort(v.begin(), v.end())` works out of the box. For custom `struct` `s`, you **must** supply a comparator (lambda or `operator<` overload) — there's no default.

9. Sorting + DSA Patterns

The core question to ask: “Does the order of elements matter for correctness, or just their values/relationships? If just relationships, sorting first often turns an $O(n^2)$ brute force into $O(n \log n)$.”

Sorting + Two Pointers

Once sorted, you can use two pointers from both ends to find pairs/triplets summing to a target, since sorted order lets you decide which pointer to move based on whether the sum is too big or too small.

Two Sum (sorted array variant):

```
vector<int> twoSum(vector<int>& arr, int target) {
    sort(arr.begin(), arr.end());
    int i = 0, j = arr.size() - 1;
    while (i < j) {
        int sum = arr[i] + arr[j];
        if (sum == target) return {arr[i], arr[j]};
        else if (sum < target) i++;
        else j--;
    }
    return {};
}
```

3Sum — fix one element, two-pointer the rest:

```
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> res;
    for (int i = 0; i < (int)nums.size(); i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicates
        int j = i + 1, k = nums.size() - 1;
        while (j < k) {
            int sum = nums[i] + nums[j] + nums[k];
            if (sum == 0) {
                res.push_back({nums[i], nums[j], nums[k]});
                j++; k--;
                while (j < k && nums[j] == nums[j - 1]) j++;
                while (j < k && nums[k] == nums[k + 1]) k--;
            } else if (sum < 0) j++;
            else k--;
        }
    }
    return res;
}
```

4Sum — same idea, one more fixed loop before the two-pointer scan (fix 2 elements, two-pointer the rest). $O(n^3)$.

Sorting + Greedy

Sorting by a key (deadline, end time, ratio) often makes the **greedy choice provably optimal**.

- **Activity Selection:** sort by end time, greedily pick the next activity whose start time $\geq$ last picked activity's end time.
- **Minimum Platforms:** sort arrival and departure times separately, use two pointers to track overlapping trains at any moment.

Sorting + Intervals

Sort by start time first — this is the universal first step for interval problems.

- **Merge Intervals:** sort by start, merge overlapping intervals in one linear pass.
- **Meeting Rooms:** sort by start, check if any interval's start < previous interval's end (overlap $\rightarrow$ can't attend all).
- **Insert Interval:** since intervals are already sorted, binary search or linear scan to find where the new interval fits, then merge overlaps.

Sorting + Binary Search

Binary search **requires** sorted data. Common combo: sort, then binary search for exact values, boundaries (lower_bound/upper_bound), or use binary search on the **answer space** (e.g., “minimum capacity to ship packages in D days”).

- **Search in sorted data:** `binary_search()`, `lower_bound()`, `upper_bound()`.
- **Kth element problems:** sort + direct index access `arr[k-1]`, OR avoid full sort with Quickselect/heap for better complexity.

Sorting + Hashing

Sometimes sorting + a hash set/map together solve problems neither could alone efficiently.

- **Duplicate detection:** sort then check adjacent elements ($O(n \log n)$, $O(1)$ space) vs. hash set ($O(n)$ time, $O(n)$ space) — know both trade-offs.
- **Frequency problems:** hash map to count frequencies, then sort by frequency (e.g., “Top K Frequent Elements”, “Sort Characters by Frequency”).

Interview Tip: When you see “find pairs/triplets with sum X”, “minimum/maximum number of resources needed for overlapping ranges”, or “Kth largest/smallest” — say out loud: *“I’ll sort first, since that unlocks a two-pointer / greedy / binary-search technique.”* This shows structured thinking, not guessing.

10. Important Advanced Sorting Problems

Count Inversions

Problem: Count pairs (i, j) where $i < j$ but $arr[i] > arr[j]$ (measures “how far” an array is from sorted).

Brute Force — $O(n^2)$

```
long long countInversionsBrute(vector<int>& arr) {
    long long count = 0;
    int n = arr.size();
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (arr[i] > arr[j]) count++;
    return count;
}
```

Merge Sort $O(n \log n)$ Solution

Key insight: During the merge step, whenever we take an element from the **right** half before finishing the **left** half, every remaining element in the left half is greater than it — that’s $(mid - left + 1)$ inversions in one shot.

```
long long merge(vector<int>& arr, int low, int mid, int high) {
    vector<int> temp;
    int left = low, right = mid + 1;
    long long invCount = 0;

    while (left <= mid && right <= high) {
        if (arr[left] <= arr[right]) {
            temp.push_back(arr[left++]);
        } else {
            invCount += (mid - left + 1); // KEY LINE: count inversions
            temp.push_back(arr[right++]);
        }
    }
    while (left <= mid) temp.push_back(arr[left++]);
    while (right <= high) temp.push_back(arr[right++]);

    for (int i = low; i <= high; i++) arr[i] = temp[i - low];
    return invCount;
}

long long countInversions(vector<int>& arr, int low, int high) {
    long long count = 0;
    if (low >= high) return 0;
    int mid = low + (high - low) / 2;
    count += countInversions(arr, low, mid);
    count += countInversions(arr, mid + 1, high);
    count += merge(arr, low, mid, high);
    return count;
}
```

Dry Run

```
Array: [2, 4, 1, 3, 5]
Split -> [2,4] [1,3,5]
Merge [2,4] with [1,3,5]:
    left=2, right=1 -> 1<2 -> invCount += (mid-left+1)=2 -> take 1
    left=2, right=3 -> 2<3 -> take 2
    left=4, right=3 -> 3<4 -> invCount += 1 -> take 3
    left=4, right=5 -> take 4, then 5
Total inversions = 3    ( (2,1),(4,1),(4,3) )
```

Time: $O(n \log n)$, Space: $O(n)$.

Kth Largest Element — Compare All Approaches

Approach	Time	Space	Notes

Sort descending, index <code>k-1</code>	$O(n \log n)$	$O(1)$ extra	Simplest, wasteful if only k is needed
Max Heap (heapify all, pop k times)	$O(n + k \log n)$	$O(n)$	Good general approach
Min Heap of size k	$O(n \log k)$	$O(k)$	Best when $k \ll n$
Quickselect	$O(n)$ average, $O(n^2)$ worst	$O(1)$	Fastest average case

Min Heap of size k (C++):

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap; // min-heap
    for (int num : nums) {
        minHeap.push(num);
        if (minHeap.size() > k) minHeap.pop(); // keep only top k largest
    }
    return minHeap.top();
}
```

Kth Smallest Element

Same approaches mirrored: sort ascending, **max heap of size k** , or Quickselect looking for the $(n-k)$ -th largest / k -th smallest directly.

```
int findKthSmallest(vector<int>& nums, int k) {
    priority_queue<int> maxHeap; // max-heap of size k
    for (int num : nums) {
        maxHeap.push(num);
        if (maxHeap.size() > k) maxHeap.pop();
    }
    return maxHeap.top();
}
```

Quickselect (from scratch)

Idea: Like Quick Sort, but only recurse into the ONE side that contains the target index — skip the other side entirely. This gives average $O(n)$ instead of $O(n \log n)$.

- **Partition:** same Lomuto partition as Quick Sort.
- **Pivot:** choose randomly to avoid worst case on adversarial/sorted input.
- **Search only one side:** after partitioning, compare the pivot's final index to the target index k , and recurse only into the half that contains it.

```
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

int quickSelect(vector<int>& arr, int low, int high, int k) {
    // find element that WOULD be at index k in sorted order
    if (low == high) return arr[low];

    int randIdx = low + rand() % (high - low + 1);
    swap(arr[randIdx], arr[high]);
    int pivotIdx = partition(arr, low, high);

    if (pivotIdx == k) return arr[pivotIdx];
    else if (pivotIdx < k) return quickSelect(arr, pivotIdx + 1, high, k);
    else return quickSelect(arr, low, pivotIdx - 1, k);
}

// Kth largest = index (n - k) in 0-indexed sorted ascending order
int findKthLargestQuickSelect(vector<int>& nums, int k) {
    int n = nums.size();
    return quickSelect(nums, 0, n - 1, n - k);
}
```

```
    return quickSelect(nums, 0, n - 1, n - k);  
}
```

Dry Run (Kth largest = 2nd largest, target index = n-k)

Array: [3,2,1,5,6,4], k=2 -> target index = 6-2 = 4

Partition around random pivot (say 4, moved to end)... eventually narrows down until pivotIdx == 4 -> return arr[4] = 5 (2nd largest)

Complexity

- Average: **$O(n)$** (each partition roughly halves the search space, geometric series sums to $O(n)$).
- Worst: **$O(n^2)$** (bad pivot every time) — mitigated with randomized pivot selection.
- Space: $O(1)$ extra (in-place), $O(\log n)$ recursion in practice.

Key Idea: Reuse Quick Sort's partition, but discard the irrelevant half instead of recursing into both. **When to Use:**

Whenever you need only the Kth smallest/largest (or top-k unordered), not a fully sorted array. **Interview Tip:** Always mention the random pivot — interviewers specifically probe whether you know Quickselect degrades to $O(n^2)$ without it.

11. Important LeetCode-Style Problems

For each problem: Problem, Example, Brute Force, Optimized Approach, Why It Works, C++ Code, Dry Run notes, Complexity.

1. Sort an Array (LC 912)

Problem: Sort an integer array in ascending order without using built-in sort. **Example:** [5,2,3,1] → [1,2,3,5] **Brute force:** Bubble/Selection Sort — $O(n^2)$. **Optimized:** Merge Sort — guarantees $O(n \log n)$.

```
class Solution {
public:
    void merge(vector<int>& a, int l, int m, int h) {
        vector<int> t; int i=l, j=m+1;
        while (i<=m && j<=h) t.push_back(a[i]<=a[j]?a[i++]:a[j++]);
        while (i<=m) t.push_back(a[i++]);
        while (j<=h) t.push_back(a[j++]);
        for (int k=l;k<=h;k++) a[k]=t[k-l];
    }
    void ms(vector<int>& a, int l, int h) {
        if (l>=h) return;
        int m=l+(h-l)/2;
        ms(a,l,m); ms(a,m+1,h); merge(a,l,m,h);
    }
    vector<int> sortArray(vector<int>& nums) {
        ms(nums, 0, nums.size()-1);
        return nums;
    }
};
```

Complexity: Time $O(n \log n)$, Space $O(n)$.

2-4. Bubble / Selection / Insertion Sort

Covered fully with code in Section 2. **Complexity:** $O(n^2)$ time ($O(n)$ best for Insertion/Bubble), $O(1)$ space.

5. Merge Sorted Array (LC 88)

Problem: Merge `nums2` into `nums1` in-place; `nums1` has extra trailing space (size `m+n`). **Example:** `nums1=[1,2,3,0,0,0]`, `m=3`, `nums2=[2,5,6]`, `n=3` → [1,2,2,3,5,6] **Brute force:** Merge into a new array, copy back — $O(m+n)$ time, $O(m+n)$ space. **Optimized:** Merge from the back to avoid overwriting unread elements — $O(1)$ extra space.

```
class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;
        while (j >= 0) {
            if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
            else nums1[k--] = nums2[j--];
        }
    }
};
```

Why it works: Filling from the back means we only ever overwrite already-copied/unneeded slots. **Complexity:** Time $O(m+n)$, Space $O(1)$.

6. Sort Colors (LC 75, Dutch National Flag)

Problem: Sort an array of 0s, 1s, 2s in one pass, in-place. **Example:** [2,0,2,1,1,0] → [0,0,1,1,2,2] **Brute force:** Counting sort (two passes) — $O(n)$ time but two passes. **Optimized:** Three pointers (`low`, `mid`, `high`) — one pass.

```
class Solution {
public:
    void sortColors(vector<int>& nums) {
        int low = 0, mid = 0, high = nums.size() - 1;
        while (mid <= high) {
            if (nums[mid] == 0) swap(nums[low++], nums[mid++]);
            else if (nums[mid] == 1) mid++;
            else if (nums[mid] == 2) swap(nums[mid], nums[high--]);
        }
    }
};
```

```

        else if (nums[mid] == 1) mid++;
        else swap(nums[mid], nums[high--]);
    }
}
};

```

Why it works: Maintains 3 regions: $[0, low) = 0s$, $[low, mid) = 1s$, $[high+1, n) = 2s$; mid scans unknowns. **Complexity:** Time $O(n)$, Space $O(1)$.

7. Merge Intervals (LC 56)

Problem: Merge all overlapping intervals. **Example:** $[[1,3],[2,6],[8,10],[15,18]] \rightarrow [[1,6],[8,10],[15,18]]$ **Brute force:** Compare every pair — $O(n^2)$. **Optimized:** Sort by start, merge in one linear pass.

```

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        vector<vector<int>> res;
        for (auto& iv : intervals) {
            if (!res.empty() && iv[0] <= res.back()[1])
                res.back()[1] = max(res.back()[1], iv[1]);
            else
                res.push_back(iv);
        }
        return res;
    }
};

```

Complexity: Time $O(n \log n)$, Space $O(n)$.

8. Insert Interval (LC 57)

Problem: Insert a new interval into a sorted, non-overlapping list, merging as needed.

```

class Solution {
public:
    vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>& newInterval) {
        vector<vector<int>> res;
        int i = 0, n = intervals.size();
        while (i < n && intervals[i][1] < newInterval[0]) res.push_back(intervals[i++]);
        while (i < n && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = min(newInterval[0], intervals[i][0]);
            newInterval[1] = max(newInterval[1], intervals[i][1]);
            i++;
        }
        res.push_back(newInterval);
        while (i < n) res.push_back(intervals[i++]);
        return res;
    }
};

```

Complexity: Time $O(n)$, Space $O(n)$.

9. 3Sum (LC 15)

Full explanation and code already given in Section 9 (Sorting + Two Pointers). **Complexity:** Time $O(n^2)$, Space $O(1)$ extra (excluding output).

10. 4Sum (LC 18)

```

class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        sort(nums.begin(), nums.end());
        vector<vector<int>> res;
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            if (i > 0 && nums[i] == nums[i-1]) continue;
            for (int j = i+1; j < n; j++) {

```

```

        if (j > i+1 && nums[j] == nums[j-1]) continue;
        int k = j+1, l = n-1;
        while (k < l) {
            long long sum = (long long)nums[i]+nums[j]+nums[k]+nums[l];
            if (sum == target) {
                res.push_back({nums[i], nums[j], nums[k], nums[l]});
                k++; l--;
                while (k < l && nums[k] == nums[k-1]) k++;
                while (k < l && nums[l] == nums[l+1]) l--;
            } else if (sum < target) k++;
            else l--;
        }
    }
}
return res;
};

```

Complexity: Time $O(n^3)$, Space $O(1)$ extra.

11. Kth Largest Element (LC 215)

Full explanation in Section 10. Use min-heap of size k or Quickselect. **Complexity:** $O(n \log k)$ heap, $O(n)$ average Quickselect.

12. Kth Smallest Element

Same pattern with max-heap of size k . **Complexity:** $O(n \log k)$.

13. Sort Characters by Frequency (LC 451)

Problem: Sort characters of a string by decreasing frequency. **Example:** "tree" → "eert" or "eetr"

```

class Solution {
public:
    string frequencySort(string s) {
        unordered_map<char, int> freq;
        for (char c : s) freq[c]++;

        vector<pair<char, int>> v(freq.begin(), freq.end());
        sort(v.begin(), v.end(), [](auto& a, auto& b) {
            return a.second > b.second;
        });

        string res;
        for (auto& [ch, cnt] : v)
            res += string(cnt, ch);
        return res;
    }
};

```

Why it works: Hashing collects frequencies, sorting orders by frequency descending. **Complexity:** Time $O(n + k \log k)$ where k = distinct chars, Space $O(n)$.

14. Largest Number (LC 179)

Problem: Arrange numbers (as strings) to form the largest possible concatenated number. **Example:** [3,30,34,5,9] → "9534330" **Key idea:** Custom comparator: $a+b > b+a$ (string concatenation comparison).

```

class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> nums_str;
        for (int n : nums) nums_str.push_back(to_string(n));

        sort(nums_str.begin(), nums_str.end(), [](const string& a, const string& b) {
            return a + b > b + a; // key trick
        });

        if (nums_str[0] == "0") return "0"; // all zeros case
    }
};

```

```

        string res;
        for (auto& s : nums_str) res += s;
        return res;
    }
};

```

Why it works: Comparing `a+b` vs `b+a` directly tests which order produces a bigger number for that pair — this comparator is transitive, giving a valid strict weak ordering. **Complexity:** Time $O(n \log n \cdot x)$ where x = avg string length (for the string comparisons), Space $O(n)$.

15. Count Inversions

Full explanation and code in Section 10. **Complexity:** $O(n \log n)$.

16. Reverse Pairs (LC 493)

Problem: Count pairs (i, j) where $i < j$ and $arr[i] > 2 \cdot arr[j]$. **Key idea:** Modified merge sort — count reverse pairs *before* merging (using a separate pointer scan), since the $2 \cdot arr[j]$ condition needs different logic than a normal inversion count.

```

class Solution {
public:
    int cnt = 0;
    void merge(vector<int>& a, int l, int m, int h) {
        int j = m + 1;
        for (int i = l; i <= m; i++) {
            while (j <= h && a[i] > 2LL * a[j]) j++;
            cnt += (j - (m + 1));
        }
        // standard merge
        vector<int> t; int x=l, y=m+1;
        while (x<=m && y<=h) t.push_back(a[x]<=a[y]?a[x++]:a[y++]);
        while (x<=m) t.push_back(a[x++]);
        while (y<=h) t.push_back(a[y++]);
        for (int k=l; k<=h; k++) a[k]=t[k-l];
    }
    void ms(vector<int>& a, int l, int h) {
        if (l>=h) return;
        int m=l+(h-l)/2;
        ms(a, l, m); ms(a, m+1, h); merge(a, l, m, h);
    }
    int reversePairs(vector<int>& nums) {
        ms(nums, 0, nums.size()-1);
        return cnt;
    }
};

```

Complexity: Time $O(n \log n)$, Space $O(n)$.

17. Meeting Rooms (LC 252)

Problem: Given intervals, determine if a person can attend all meetings (no overlaps).

```

class Solution {
public:
    bool canAttendMeetings(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());
        for (int i = 1; i < (int)intervals.size(); i++)
            if (intervals[i][0] < intervals[i-1][1]) return false;
        return true;
    }
};

```

Complexity: Time $O(n \log n)$, Space $O(1)$ extra.

18. Minimum Platforms

Problem: Given arrival and departure times of trains, find minimum platforms needed simultaneously.

```

int findMinPlatforms(vector<int> arr, vector<int> dep) {

```



```

        sort(arr.begin(), arr.end());
        sort(dep.begin(), dep.end());

        int platforms = 0, maxPlatforms = 0;
        int i = 0, j = 0, n = arr.size();

        while (i < n && j < n) {
            if (arr[i] <= dep[j]) {
                platforms++;
                i++;
            } else {
                platforms--;
                j++;
            }
            maxPlatforms = max(maxPlatforms, platforms);
        }
        return maxPlatforms;
    }
}

```

Why it works: Sorting arrivals/departures independently and sweeping with two pointers tracks how many trains are simultaneously “in the station” at any moment. **Complexity:** Time $O(n \log n)$, Space $O(1)$ extra.

19. Maximum Number of Events (LC 1353)

Problem: Attend the maximum number of events, each with a start/end day; can attend only 1 event per day. **Key idea:** Sort events by start day; use a **min-heap of end days**; each day, push all events starting that day, pop expired events, then greedily attend the event ending soonest (frees up later days for events with tighter deadlines).

```

class Solution {
public:
    int maxEvents(vector<vector<int>>& events) {
        sort(events.begin(), events.end());
        priority_queue<int, vector<int>, greater<int>> minHeap; // end days
        int i = 0, n = events.size(), day = 0, count = 0;

        while (i < n || !minHeap.empty()) {
            if (minHeap.empty()) day = events[i][0]; // jump to next event's start
            while (i < n && events[i][0] <= day) minHeap.push(events[i++][1]);
            while (!minHeap.empty() && minHeap.top() < day) minHeap.pop(); // expired
            if (!minHeap.empty()) { minHeap.pop(); count++; }
            day++;
        }
        return count;
    }
};

```

Complexity: Time $O(n \log n)$, Space $O(n)$.

20. Merge K Sorted Lists (LC 23)

Problem: Merge k sorted linked lists into one sorted list. **Brute force:** Concatenate all values, sort — $O(N \log N)$ where N = total nodes. **Optimized:** Min-heap of size k (always merge the smallest head node next).

```

struct ListNode { int val; ListNode* next; ListNode(int x): val(x), next(nullptr) {} };

class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*> lists) {
        auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);

        for (auto node : lists) if (node) pq.push(node);

        ListNode dummy(0);
        ListNode* tail = &dummy;

        while (!pq.empty()) {
            ListNode* smallest = pq.top(); pq.pop();
            tail->next = smallest;
            tail = tail->next;
            if (smallest->next) pq.push(smallest->next);
        }
        return dummy.next;
    }
}

```

```
};
```

Why it works: The heap always has the k current “frontier” nodes; popping the minimum and pushing its successor mimics a k -way merge (generalization of Merge Sort’s 2-way merge). **Complexity:** Time $O(N \log k)$, Space $O(k)$.

12. Sorting Complexity Table

Algorithm	Best	Average	Worst	Space	Stable	In-Place
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes	No
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes	No
Bucket	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes	No
Shell	$O(n \log n)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	No	Yes
Tim Sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Intro Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No	Yes
Cycle	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes

13. How to Choose a Sorting Algorithm

If array is nearly sorted → Insertion Sort ($O(n)$ best case, adaptive) or Tim Sort.

If guaranteed $O(n \log n)$ is required → Merge Sort (if stability/extra memory OK) or Heap Sort (if $O(1)$ space needed). Avoid plain Quick Sort — its worst case is $O(n^2)$.

If stable sorting is required → Merge Sort, Insertion Sort (small n), Counting/Radix/Bucket Sort (for suitable data), or `stable_sort()` in C++.

If memory is limited → Heap Sort or Quick Sort (both $O(\log n)$ or $O(1)$ extra space) — avoid Merge Sort's $O(n)$ buffer.

If integer range is small → Counting Sort — beats $O(n \log n)$ with $O(n+k)$ time.

If numbers have limited digits → Radix Sort — $O(d \cdot n)$, great for fixed-width integers/strings.

If you need Kth largest/smallest → Quickselect ($O(n)$ average) or a heap of size k ($O(n \log k)$) — don't fully sort unless you need the whole order.

If you need to sort objects using multiple conditions → Custom comparator with primary key + tie-breaker chain (Section 7), using `sort()` (if ties don't need original order) or `stable_sort()` (if they do).

14. Common C++ Mistakes

- **Incorrect loop boundaries** — e.g., `j < n - i` instead of `j < n - i - 1` in Bubble Sort, causing out-of-bounds access or missed comparisons.
- **Wrong swapping** — swapping inside the wrong loop (e.g., swapping every comparison in Selection Sort instead of only after finding the true minimum).
- **Incorrect recursion base case** — using `if (low == high)` instead of `if (low >= high)` in Merge/Quick Sort; fails to terminate or misses single-element sub-arrays when `low > high`.
- **Infinite recursion** — in Quick Sort, recursing with the same `(low, high)` bounds due to a partition function that doesn't actually shrink the range (e.g., using `pivotIdx` instead of `pivotIdx ± 1`).
- **Wrong partition implementation** — mixing up Lomuto's returned pivot index with Hoare's returned index (they require different recursive bounds — Hoare is `[low, j] / [j+1, high]`, NOT `pivotIdx ± 1`).
- **Incorrect comparator** — using `<=` in a `sort()` comparator, which violates strict weak ordering and can cause crashes or undefined behavior in `libstdc++`.
- **Comparator returning wrong condition** — writing `return a > b` when you meant ascending order (easy to flip signs and get reverse of intended order).
- **Integer overflow** — computing `mid = (low + high) / 2` can overflow for large `low / high`; always use `mid = low + (high - low) / 2`.
- **Forgetting references** — passing `vector<int> arr` by value into a sort function silently sorts a copy, leaving the caller's array untouched. Always use `vector<int>& arr`.
- **Unnecessary copying of vectors** — passing large vectors/structs by value in comparators (`bool cmp(pair<int,int> a, pair<int,int> b)`) instead of by const reference — hurts performance on large inputs.
- **Incorrect use of `sort()`** — forgetting `.begin() / .end()`, or sorting only part of the range by mistake (`sort(arr.begin(), arr.end() - 1)` unintentionally excluding the last element).
- **Using `greater<int>()`** — forgetting the parentheses (`greater<int>` is a type, `greater<int>()` constructs an instance) — a very common beginner compile error.
- **Incorrect lambda syntax** — forgetting the capture list `[]`, or forgetting `const &` on struct parameters, causing unnecessary copies or compile errors when capturing outside variables.
- **Wrong complexity calculation** — claiming Quick Sort is "always $O(n \log n)$ " (it's $O(n^2)$ worst case) or claiming `buildMaxHeap` is $O(n \log n)$ (it's actually $O(n)$, a subtle and commonly-misstated result).

15. Interview Preparation

Each question lists the **problem + hint** first, then the **solution** — cover the solution and try it yourself first.

Beginner Questions (15)

B1. Sort an array using Bubble Sort. *Hint:* Adjacent swaps, largest bubbles to the end. **Solution:** See Section 2.
Complexity: $O(n^2)$ time, $O(1)$ space.

B2. Sort an array using Selection Sort. *Hint:* Find min of remaining, swap to front. **Solution:** See Section 2.
Complexity: $O(n^2)$ time, $O(1)$ space.

B3. Sort an array using Insertion Sort. *Hint:* Insert each element into the sorted prefix. **Solution:** See Section 2.
Complexity: $O(n^2)$ worst, $O(n)$ best, $O(1)$ space.

B4. Check if an array is sorted. *Hint:* Single pass comparing adjacent elements.

```
bool isSorted(vector<int>& a) {
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i] < a[i-1]) return false;
    return true;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

B5. Reverse an array without extra space. *Hint:* Two pointers from both ends, swap and move inward.

```
void reverseArray(vector<int>& a) {
    int i = 0, j = a.size() - 1;
    while (i < j) swap(a[i++], a[j--]);
}
```

Complexity: $O(n)$ time, $O(1)$ space.

B6. Find the second largest element without full sort. *Hint:* Track largest and second-largest in one pass.

```
int secondLargest(vector<int>& a) {
    int first = INT_MIN, second = INT_MIN;
    for (int x : a) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x != first) second = x;
    }
    return second;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

B7. Merge two sorted arrays into a third array. *Hint:* Standard merge step from Merge Sort. **Solution:** See `merge()` in Section 3. **Complexity:** $O(n+m)$ time, $O(n+m)$ space.

B8. Sort an array of 0s and 1s only. *Hint:* Two pointers, or counting sort with 2 buckets.

```
void sortBinary(vector<int>& a) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        if (a[lo] == 0) lo++;
        else swap(a[lo], a[hi--]);
    }
}
```

Complexity: $O(n)$ time, $O(1)$ space.

B9. Find the missing number in an array of 1..n. *Hint:* Sum formula $n(n+1)/2$ minus actual sum, or sort + linear scan.

```
int missingNumber(vector<int>& a, int n) {
    long long expected = (long long)n * (n + 1) / 2;
    long long actual = 0;
    for (int x : a) actual += x;
    return expected - actual;
}
```

```
}
```

Complexity: $O(n)$ time, $O(1)$ space.

B10. Sort a string's characters alphabetically. *Hint:* Strings support the same iterators as vectors.

```
string sortString(string s) {  
    sort(s.begin(), s.end());  
    return s;  
}
```

Complexity: $O(n \log n)$ time.

B11. Find duplicates in an array using sorting. *Hint:* Sort, then check adjacent equal elements.

```
vector<int> findDuplicates(vector<int> a) {  
    sort(a.begin(), a.end());  
    vector<int> dups;  
    for (int i = 1; i < (int)a.size(); i++)  
        if (a[i] == a[i-1]) dups.push_back(a[i]);  
    return dups;  
}
```

Complexity: $O(n \log n)$ time, $O(1)$ extra (excluding output).

B12. Sort an array in descending order using STL. *Hint:* `greater<int>()` comparator.

```
sort(a.begin(), a.end(), greater<int>());
```

Complexity: $O(n \log n)$.

B13. Find the union of two sorted arrays. *Hint:* Two pointers, skip duplicates.

```
vector<int> unionArrays(vector<int>& a, vector<int>& b) {  
    vector<int> res; int i=0, j=0;  
    while (i<(int)a.size() && j<(int)b.size()) {  
        if (a[i] < b[j]) { if (res.empty()||res.back()!=a[i]) res.push_back(a[i]); i++; }  
        else if (b[j] < a[i]) { if (res.empty()||res.back()!=b[j]) res.push_back(b[j]); j++; }  
        else { if (res.empty()||res.back()!=a[i]) res.push_back(a[i]); i++; j++; }  
    }  
    while (i<(int)a.size()) { if (res.empty()||res.back()!=a[i]) res.push_back(a[i]); i++; }  
    while (j<(int)b.size()) { if (res.empty()||res.back()!=b[j]) res.push_back(b[j]); j++; }  
    return res;  
}
```

Complexity: $O(n+m)$ time (arrays already sorted).

B14. Find the intersection of two sorted arrays. *Hint:* Two pointers, advance the smaller one, match on equality.

```
vector<int> intersectionArrays(vector<int>& a, vector<int>& b) {  
    vector<int> res; int i=0, j=0;  
    while (i<(int)a.size() && j<(int)b.size()) {  
        if (a[i] < b[j]) i++;  
        else if (b[j] < a[i]) j++;  
        else { res.push_back(a[i]); i++; j++; }  
    }  
    return res;  
}
```

Complexity: $O(n+m)$ time.

B15. Explain, without code, why Bubble Sort is stable but Selection Sort is not. *Hint:* Think about what a swap does to equal elements in each algorithm. **Answer:** Bubble Sort only swaps strictly-greater adjacent pairs, so equal elements are never swapped past each other. Selection Sort swaps the found minimum into position `i`, which can jump it past equal elements that were originally before it — breaking relative order.

Easy Questions (15)

E1. Sort Colors (Dutch National Flag). *Hint:* Three-pointer one-pass partition. **Solution:** Section 11, Problem 6.

Complexity: $O(n)$ time, $O(1)$ space.

E2. Merge Sorted Array (in-place). *Hint:* Fill from the back. **Solution:** Section 11, Problem 5. **Complexity:** $O(m+n)$ time, $O(1)$ space.

E3. Find Kth largest element (using sort). *Hint:* Sort descending, index `k-1`.

```
int kthLargestSorted(vector<int> a, int k) {
    sort(a.rbegin(), a.rend());
    return a[k-1];
}
```

Complexity: $O(n \log n)$.

E4. Meeting Rooms (can attend all?). *Hint:* Sort by start, check adjacent overlap. **Solution:** Section 11, Problem 17. **Complexity:** $O(n \log n)$.

E5. Sort an array of squares of a sorted array (may contain negatives). *Hint:* Two pointers from both ends (largest square comes from an extreme).

```
vector<int> sortedSquares(vector<int>& a) {
    int n = a.size();
    vector<int> res(n);
    int i = 0, j = n-1, pos = n-1;
    while (i <= j) {
        int left = a[i]*a[i], right = a[j]*a[j];
        if (left > right) { res[pos--] = left; i++; }
        else { res[pos--] = right; j--; }
    }
    return res;
}
```

Complexity: $O(n)$ time, $O(n)$ space.

E6. Relative Sort Array (LC 1122). *Hint:* Count frequencies, output arr2's order first, then remaining sorted.

```
vector<int> relativeSortArray(vector<int>& a1, vector<int>& a2) {
    unordered_map<int,int> freq;
    for (int x : a1) freq[x]++;
    vector<int> res;
    for (int x : a2) { while (freq[x]-- > 0) res.push_back(x); freq.erase(x); }
    vector<int> rest;
    for (auto& [val, cnt] : freq) for (int i=0; i<cnt; i++) rest.push_back(val);
    sort(rest.begin(), rest.end());
    for (int x : rest) res.push_back(x);
    return res;
}
```

Complexity: $O(n \log n)$ time.

E7. Height Checker (LC 1051). *Hint:* Compare with a sorted copy.

```
int heightChecker(vector<int>& heights) {
    vector<int> expected = heights;
    sort(expected.begin(), expected.end());
    int count = 0;
    for (int i = 0; i < (int)heights.size(); i++)
        if (heights[i] != expected[i]) count++;
    return count;
}
```

Complexity: $O(n \log n)$.

E8. Assign Cookies (LC 455, greedy + sort). *Hint:* Sort both arrays, two-pointer greedy match.

```
int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(g.begin(), g.end()); sort(s.begin(), s.end());
    int i = 0, j = 0;
    while (i < (int)g.size() && j < (int)s.size()) {
        if (s[j] >= g[i]) i++;
        j++;
    }
    return i;
}
```


Complexity: $O(n \log n)$.

E9. Squares/Two Sum II (sorted input). *Hint:* Two pointers converge on target. **Solution:** Section 9. **Complexity:** $O(n)$.

E10. Majority Element (LC 169) using sort. *Hint:* Sorted array's middle element is always the majority element (if one exists).

```
int majorityElement(vector<int> a) {
    sort(a.begin(), a.end());
    return a[a.size()/2];
}
```

Complexity: $O(n \log n)$.

E11. Minimum Absolute Difference in an Array (LC 1200). *Hint:* Sort, min gap is always between adjacent elements.

```
vector<vector<int>> minimumAbsDifference(vector<int>& a) {
    sort(a.begin(), a.end());
    int minDiff = INT_MAX;
    for (int i = 1; i < (int)a.size(); i++) minDiff = min(minDiff, a[i]-a[i-1]);
    vector<vector<int>> res;
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i]-a[i-1] == minDiff) res.push_back({a[i-1], a[i]});
    return res;
}
```

Complexity: $O(n \log n)$.

E12. Sort an array by parity (even before odd). *Hint:* Two pointer partition like Sort Colors.

```
vector<int> sortArrayByParity(vector<int>& a) {
    int i = 0, j = a.size()-1;
    while (i < j) {
        if (a[i] % 2 == 0) i++;
        else swap(a[i], a[j--]);
    }
    return a;
}
```

Complexity: $O(n)$ time, $O(1)$ space.

E13. Valid Anagram (LC 242) using sort. *Hint:* Sort both strings and compare.

```
bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    sort(s.begin(), s.end()); sort(t.begin(), t.end());
    return s == t;
}
```

Complexity: $O(n \log n)$.

E14. Third Maximum Number (LC 414). *Hint:* Track 3 distinct max values in one pass (or sort + dedupe).

```
int thirdMax(vector<int>& nums) {
    set<int, greater<int>> top3;
    for (int n : nums) {
        top3.insert(n);
        if (top3.size() > 3) top3.erase(prev(top3.end()));
    }
    return top3.size() == 3 ? *next(top3.begin(), 2) : *top3.begin();
}
```

Complexity: $O(n \log 3) \approx O(n)$.

E15. Sort a nearly sorted (k-sorted) array. *Hint:* Min-heap of size $k+1$.

```
vector<int> sortKSorted(vector<int>& a, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        minHeap.push(a[i]);
        if ((int)minHeap.size() > k) { res.push_back(minHeap.top()); minHeap.pop(); }
    }
}
```

```

while (!minHeap.empty()) { res.push_back(minHeap.top()); minHeap.pop(); }
return res;
}

```

Complexity: $O(n \log k)$ time, $O(k)$ space.

Medium Questions (20)

M1. 3Sum. *Hint:* Sort + fix one + two-pointer. **Solution:** Section 9. **Complexity:** $O(n^2)$.

M2. 4Sum. *Hint:* Sort + fix two + two-pointer. **Solution:** Section 11, Problem 10. **Complexity:** $O(n^3)$.

M3. Merge Intervals. *Hint:* Sort by start, merge linearly. **Solution:** Section 11, Problem 7. **Complexity:** $O(n \log n)$.

M4. Insert Interval. *Hint:* Linear scan since already sorted. **Solution:** Section 11, Problem 8. **Complexity:** $O(n)$.

M5. Kth Largest Element (heap/quickselect). *Hint:* Min-heap of size k or Quickselect. **Solution:** Section 10. **Complexity:** $O(n \log k)$ or $O(n)$ average.

M6. Sort Characters By Frequency. *Hint:* Hash map + sort by frequency. **Solution:** Section 11, Problem 13. **Complexity:** $O(n + k \log k)$.

M7. Largest Number. *Hint:* Custom comparator $a+b > b+a$. **Solution:** Section 11, Problem 14. **Complexity:** $O(n \log n \cdot x)$.

M8. Meeting Rooms II (min meeting rooms needed). *Hint:* Separate sorted start/end arrays, two-pointer sweep (like Minimum Platforms).

```

int minMeetingRooms(vector<vector<int>>& intervals) {
    vector<int> starts, ends;
    for (auto& iv : intervals) { starts.push_back(iv[0]); ends.push_back(iv[1]); }
    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());

    int rooms = 0, maxRooms = 0, i = 0, j = 0;
    while (i < (int)starts.size()) {
        if (starts[i] < ends[j]) { rooms++; i++; }
        else { rooms--; j++; }
        maxRooms = max(maxRooms, rooms);
    }
    return maxRooms;
}

```

Complexity: $O(n \log n)$.

M9. Non-overlapping Intervals (LC 435) — min removals to make non-overlapping. *Hint:* Sort by end time, greedily keep intervals that don't overlap with the last kept one.

```

int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b){ return a[1] < b[1]; });
    int count = 0, lastEnd = INT_MIN;
    for (auto& iv : intervals) {
        if (iv[0] >= lastEnd) lastEnd = iv[1];
        else count++;
    }
    return count;
}

```

Complexity: $O(n \log n)$.

M10. Top K Frequent Elements (LC 347). *Hint:* Hash map for frequency, min-heap of size k or bucket sort by frequency.

```

vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int,int> freq;
    for (int n : nums) freq[n]++;

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> minHeap;
    for (auto& [val, cnt] : freq) {
        minHeap.push({cnt, val});
        if ((int)minHeap.size() > k) minHeap.pop();
    }
    vector<int> res;
    while (!minHeap.empty()) { res.push_back(minHeap.top().second); minHeap.pop(); }
}

```

```

        return res;
    }
}

```

Complexity: $O(n \log k)$.

M11. Kth Smallest Element in a Sorted Matrix (LC 378). *Hint:* Binary search on value range, count elements $\leq$ mid using sorted rows.

```

int kthSmallest(vector<vector<int>>& matrix, int k) {
    int n = matrix.size();
    int lo = matrix[0][0], hi = matrix[n-1][n-1];
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        int count = 0, row = n-1, col = 0;
        while (row >= 0 && col < n) {
            if (matrix[row][col] <= mid) { count += row + 1; col++; }
            else row--;
        }
        if (count < k) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

```

Complexity: $O(n \log(\max - \min))$.

M12. Sort a Linked List (Merge Sort on linked list, LC 148). *Hint:* Find middle with slow/fast pointers, recursively sort halves, merge.

```

ListNode* getMid(ListNode* head) {
    ListNode* slow = head, *fast = head->next;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}

ListNode* mergeTwo(ListNode* a, ListNode* b) {
    ListNode dummy(0); ListNode* t = &dummy;
    while (a && b) {
        if (a->val <= b->val) { t->next = a; a = a->next; }
        else { t->next = b; b = b->next; }
        t = t->next;
    }
    t->next = a ? a : b;
    return dummy.next;
}

ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* mid = getMid(head);
    ListNode* right = mid->next;
    mid->next = nullptr;
    return mergeTwo(sortList(head), sortList(right));
}

```

Complexity: $O(n \log n)$ time, $O(\log n)$ recursion stack space.

M13. Find Right Interval (LC 436). *Hint:* Sort start points, binary search (lower_bound) for each end value.

```

vector<int> findRightInterval(vector<vector<int>>& intervals) {
    int n = intervals.size();
    vector<pair<int,int>> starts;
    for (int i = 0; i < n; i++) starts.push_back({intervals[i][0], i});
    sort(starts.begin(), starts.end());

    vector<int> res(n, -1);
    for (int i = 0; i < n; i++) {
        int end = intervals[i][1];
        auto it = lower_bound(starts.begin(), starts.end(), make_pair(end, INT_MIN));
        if (it != starts.end()) res[i] = it->second;
    }
    return res;
}

```

Complexity: $O(n \log n)$.

M14. Sort Array By Increasing Frequency, tie by value descending (LC 1636). *Hint:* Custom comparator with frequency as primary key, value as tie-breaker.

```
vector<int> frequencySort(vector<int>& nums) {
    unordered_map<int,int> freq;
    for (int n : nums) freq[n]++;
    sort(nums.begin(), nums.end(), [&](int a, int b) {
        if (freq[a] != freq[b]) return freq[a] < freq[b];
        return a > b;
    });
    return nums;
}
```

Complexity: $O(n \log n)$.

M15. Wiggle Sort (LC 280) — arrange so $a[0] < a[1] > a[2] < a[3] \dots$ *Hint:* Sort then swap adjacent pairs, OR one-pass greedy swap without sorting.

```
void wiggleSort(vector<int>& nums) {
    for (int i = 1; i < (int)nums.size(); i++) {
        if ((i % 2 == 1 && nums[i] < nums[i-1]) ||
            (i % 2 == 0 && nums[i] > nums[i-1]))
            swap(nums[i], nums[i-1]);
    }
}
```

Complexity: $O(n)$ (greedy version) — better than the $O(n \log n)$ sort-based approach.

M16. Largest Rectangle Formed by sorting bar heights conceptually / Boats to Save People (LC 881). *Hint:* Sort weights, two pointers pairing heaviest with lightest.

```
int numRescueBoats(vector<int>& people, int limit) {
    sort(people.begin(), people.end());
    int i = 0, j = people.size() - 1, boats = 0;
    while (i <= j) {
        if (people[i] + people[j] <= limit) i++;
        j--;
        boats++;
    }
    return boats;
}
```

Complexity: $O(n \log n)$.

M17. Car Pooling (LC 1094) — sort by pickup, sweep passenger count. *Hint:* Create (location, delta) events, sort by location, sweep.

```
bool carPooling(vector<vector<int>>& trips, int capacity) {
    vector<pair<int,int>> events; // {location, capacity change}
    for (auto& t : trips) {
        events.push_back({t[1], t[0]});
        events.push_back({t[2], -t[0]});
    }
    sort(events.begin(), events.end());
    int cur = 0;
    for (auto& [loc, delta] : events) {
        cur += delta;
        if (cur > capacity) return false;
    }
    return true;
}
```

Complexity: $O(n \log n)$.

M18. Custom Sort String (LC 791). *Hint:* Count frequencies, output according to `order`, then leftovers.

```
string customSortString(string order, string s) {
    vector<int> freq(26, 0);
    for (char c : s) freq[c - 'a']++;

    string res;
    for (char c : order) {
        while (freq[c - 'a']-- > 0) res += c;
    }
    for (char c = 'a'; c <= 'z'; c++)
        while (freq[c - 'a']-- > 0) res += c;
    return res;
}
```

```
}
```

Complexity: $O(n + 26)$.

M19. Minimum Number of Arrows to Burst Balloons (LC 452). *Hint:* Sort by end coordinate, greedy — shoot at the end of the first unburst balloon.

```
int findMinArrowShots(vector<vector<int>>& points) {
    if (points.empty()) return 0;
    sort(points.begin(), points.end(), [](auto& a, auto& b) { return a[1] < b[1]; });
    int arrows = 1;
    long long end = points[0][1];
    for (auto& p : points) {
        if (p[0] > end) { arrows++; end = p[1]; }
    }
    return arrows;
}
```

Complexity: $O(n \log n)$.

M20. Sort a nearly-sorted array using Insertion Sort variant / Pancake Sort (LC 969, conceptual). *Hint:* Repeatedly flip the largest unsorted element to the front, then flip it into its final position.

```
vector<int> pancakeSort(vector<int>& arr) {
    vector<int> res;
    int n = arr.size();
    for (int size = n; size > 1; size--) {
        int maxIdx = max_element(arr.begin(), arr.begin() + size) - arr.begin();
        if (maxIdx != size - 1) {
            reverse(arr.begin(), arr.begin() + maxIdx + 1);
            res.push_back(maxIdx + 1);
            reverse(arr.begin(), arr.begin() + size);
            res.push_back(size);
        }
    }
    return res;
}
```

Complexity: $O(n^2)$ time (each pass is $O(n)$, n passes).

Hard Questions (10)

H1. Count Inversions. *Hint:* Modified merge sort, count during merge. **Solution:** Section 10. **Complexity:** $O(n \log n)$.

H2. Reverse Pairs. *Hint:* Modified merge sort with a pre-merge counting pass. **Solution:** Section 11, Problem 16. **Complexity:** $O(n \log n)$.

H3. Merge K Sorted Lists. *Hint:* Min-heap of k frontier nodes. **Solution:** Section 11, Problem 20. **Complexity:** $O(N \log k)$.

H4. Median of Two Sorted Arrays (LC 4). *Hint:* Binary search on the smaller array to find the correct partition point in $O(\log(\min(m, n)))$.

```
double findMedianSortedArrays(vector<int>& a, vector<int>& b) {
    if (a.size() > b.size()) return findMedianSortedArrays(b, a);
    int n = a.size(), m = b.size();
    int lo = 0, hi = n;

    while (lo <= hi) {
        int cutA = (lo + hi) / 2;
        int cutB = (n + m + 1) / 2 - cutA;

        int leftA = (cutA == 0) ? INT_MIN : a[cutA - 1];
        int rightA = (cutA == n) ? INT_MAX : a[cutA];
        int leftB = (cutB == 0) ? INT_MIN : b[cutB - 1];
        int rightB = (cutB == m) ? INT_MAX : b[cutB];

        if (leftA <= rightB && leftB <= rightA) {
            if ((n + m) % 2 == 0)
                return (max(leftA, leftB) + min(rightA, rightB)) / 2.0;
            else
                return max(leftA, leftB);
        } else if (leftA > rightB) hi = cutA - 1;
        else lo = cutA + 1;
    }
}
```

```

        return -1.0;
    }
}

```

Complexity: $O(\log(\min(n,m)))$ time, $O(1)$ space.

H5. Count of Smaller Numbers After Self (LC 315). *Hint:* Modified merge sort — count elements from right half placed before each left-half element (like inversions, but tracked per index).

```

class Solution {
public:
    vector<int> counts;
    vector<int> merge(vector<pair<int,int>>& idxVal, int l, int m, int h) {
        vector<pair<int,int>> temp;
        int i=l, j=m+1;
        while (i<=m && j<=h) {
            if (idxVal[i].first <= idxVal[j].first) {
                counts[idxVal[i].second] += (j - (m+1));
                temp.push_back(idxVal[i++]);
            } else temp.push_back(idxVal[j++]);
        }
        while (i<=m) { counts[idxVal[i].second] += (j - (m+1)); temp.push_back(idxVal[i++]); }
        while (j<=h) temp.push_back(idxVal[j++]);
        for (int k=l; k<=h; k++) idxVal[k]=temp[k-l];
        return {};
    }
    void ms(vector<pair<int,int>>& idxVal, int l, int h) {
        if (l>=h) return;
        int m=l+(h-l)/2;
        ms(idxVal, l, m); ms(idxVal, m+1, h); merge(idxVal, l, m, h);
    }
    vector<int> countSmaller(vector<int>& nums) {
        int n = nums.size();
        counts.assign(n, 0);
        vector<pair<int,int>> idxVal;
        for (int i=0; i<n; i++) idxVal.push_back({nums[i], i});
        ms(idxVal, 0, n-1);
        return counts;
    }
};

```

Complexity: $O(n \log n)$ time, $O(n)$ space.

H6. Maximum Gap (LC 164) — max gap between sorted adjacent elements, $O(n)$ required. *Hint:* Bucket sort (pigeonhole) using the guaranteed minimum possible gap as bucket width.

```

int maximumGap(vector<int>& nums) {
    int n = nums.size();
    if (n < 2) return 0;
    int mn = *min_element(nums.begin(), nums.end());
    int mx = *max_element(nums.begin(), nums.end());
    if (mx == mn) return 0;

    int bucketSize = max(1, (mx - mn) / (n - 1));
    int bucketCount = (mx - mn) / bucketSize + 1;
    vector<int> bucketMin(bucketCount, INT_MAX), bucketMax(bucketCount, INT_MIN);

    for (int x : nums) {
        int idx = (x - mn) / bucketSize;
        bucketMin[idx] = min(bucketMin[idx], x);
        bucketMax[idx] = max(bucketMax[idx], x);
    }

    int maxGap = 0, prevMax = mn;
    for (int i = 0; i < bucketCount; i++) {
        if (bucketMin[i] == INT_MAX) continue; // empty bucket
        maxGap = max(maxGap, bucketMin[i] - prevMax);
        prevMax = bucketMax[i];
    }
    return maxGap;
}

```

Why it works: By pigeonhole principle, the maximum gap can never occur *within* a bucket (bucket width $\leq$ average gap) — only *between* buckets, so we only need each bucket's min/max. **Complexity:** $O(n)$ time, $O(n)$ space.

H7. Count of Range Sum (LC 327) — count subarray sums within [lower, upper]. *Hint:* Prefix sums + modified merge sort (count valid j range for each i during merge).

```

class Solution {
public:
    int count = 0;
    int lower, upper;
    void merge(vector<long long>& sums, int l, int m, int h) {
        int j = m+1, k = m+1;
        for (int i = l; i <= h; i++) {
            while (j <= h && sums[j] - sums[i] < lower) j++;
            while (k <= h && sums[k] - sums[i] <= upper) k++;
            count += (k - j);
        }
        inplace_merge(sums.begin()+l, sums.begin()+m+1, sums.begin()+h+1);
    }
    void ms(vector<long long>& sums, int l, int h) {
        if (l >= h) return;
        int m = l + (h-l)/2;
        ms(sums, l, m); ms(sums, m+1, h); merge(sums, l, m, h);
    }
    int countRangeSum(vector<int>& nums, int lower, int upper) {
        this->lower = lower; this->upper = upper;
        vector<long long> prefix(nums.size()+1, 0);
        for (int i = 0; i < (int)nums.size(); i++) prefix[i+1] = prefix[i] + nums[i];
        ms(prefix, 0, prefix.size()-1);
        return count;
    }
};

```

Complexity: $O(n \log n)$ time, $O(n)$ space.

H8. Sliding Window Median (LC 480). *Hint:* Two heaps (max-heap for lower half, min-heap for upper half) with lazy deletion as the window slides.

```

// Conceptual outline (full lazy-deletion bookkeeping omitted for brevity):
// maxHeap = lower half (size k/2 or k/2+1), minHeap = upper half.
// On each slide: remove outgoing element (mark for lazy deletion), insert incoming element,
// rebalance heaps so their sizes differ by at most 1, median = top of larger heap
// (or average of both tops if sizes are equal).

```

Complexity: $O(n \log k)$ time, $O(k)$ space.

H9. Sort a huge external file that doesn't fit in memory (External Merge Sort, systems question). *Hint:* Split into memory-sized sorted chunks, k-way merge with a min-heap.

```

// Conceptual: 1) Read file in chunks of size M (fits RAM), sort each chunk in-memory
//              (e.g., std::sort), write sorted chunk to disk.
//              2) Open all chunk files, use a min-heap of size k (number of chunks)
//              holding the current front element of each chunk.
//              3) Repeatedly pop the min, write to output, read next element from
//              that chunk's file into the heap.

```

Complexity: $O(N \log(N/M))$ time (M = memory size), $O(M)$ space in RAM at a time.

H10. Find K Closest Elements (LC 658) using sorted-array binary search. *Hint:* Binary search for the left boundary of the k-length window with minimum spread.

```

vector<int> findClosestElements(vector<int>& arr, int k, int x) {
    int lo = 0, hi = arr.size() - k;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (x - arr[mid] > arr[mid + k] - x) lo = mid + 1;
        else hi = mid;
    }
    return vector<int>(arr.begin() + lo, arr.begin() + lo + k);
}

```

Complexity: $O(\log(n-k) + k)$ time.

16. Revision Cheat Sheet (10-15 Minute Review)

Algorithm	Core Idea	Best	Average	Worst	Space	Stable	In-Place	When to Use
Bubble	Swap adjacent out-of-order pairs	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Teaching only
Selection	Pick min of remaining, place front	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes	Minimize swaps
Insertion	Insert into sorted prefix	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Nearly sorted / small n
Merge	Divide, sort halves, merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	Guaranteed $O(n \log n)$, stability, linked lists
Quick	Partition around pivot, recurse	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes	General fast in-memory sort
Heap	Build heap, repeatedly extract max	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes	Guaranteed $O(n \log n)$, $O(1)$ space
Counting	Frequency array + prefix sum	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes	No	Small integer range
Radix	Sort digit by digit (LSD)	$O(dn)$	$O(dn)$	$O(dn)$	$O(n+k)$	Yes	No	Fixed-digit integers
Bucket	Distribute into buckets, sort each	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes	No	Uniform distribution
Shell	Insertion sort with shrinking gaps	$O(n \log n)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	No	Yes	Better than plain insertion
Tim Sort	Runs + insertion + merge hybrid	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	Real-world data (Python/Java default)
Intro Sort	Quick + Heap + Insertion hybrid	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No	Yes	C++ <code>std::sort</code> default

STL Quick Reference: - `sort()` → $O(n \log n)$, not stable (Intro Sort). - `stable_sort()` → $O(n \log n)$, stable (Merge Sort based). - `partial_sort(first, first+k, last)` → $O(n \log k)$, first k sorted. - `nth_element(first, first+k, last)` → $O(n)$ avg, k th element correct, rest unordered.

Comparator rule: must define strict weak ordering — never `<=`, must be consistent and transitive.

Decision shortcuts: - Nearly sorted → Insertion / Tim Sort. - Need guaranteed $O(n \log n)$ + stable → Merge Sort. - Need guaranteed $O(n \log n)$ + $O(1)$ space → Heap Sort. - Small integer range → Counting Sort. - Fixed digit-length integers → Radix Sort. - Just need k th element → Quickselect $O(n)$ avg, or heap of size k $O(n \log k)$. - Multiple sort keys → chained comparator (primary, then tie-breakers).

17. Learning Order (and Why)

1. **Bubble Sort** — simplest possible sort; builds comparison/swap intuition.
2. **Selection Sort** — introduces “find extremum, place it” pattern; contrast with Bubble Sort’s stability.
3. **Insertion Sort** — introduces the “grow a sorted region” pattern; sets up Tim Sort intuition.
4. **Merge Sort** — introduces divide-and-conquer and recursion trees; the merge step reused everywhere (inversions, k-way merge).
5. **Quick Sort** — builds on divide-and-conquer with in-place partitioning; introduces randomization to avoid worst case.
6. **Heap Sort** — introduces heaps (a structure reused constantly in problems: Kth largest, scheduling, Dijkstra); shows an alternative $O(n \log n)$ with $O(1)$ space.
7. **Counting Sort** — first non-comparison sort; shows that $O(n \log n)$ isn’t a hard limit under extra assumptions.
8. **Radix Sort** — builds directly on Counting Sort; teaches digit-by-digit thinking.
9. **Bucket Sort** — completes the non-comparison trio; teaches distribution-based thinking.
10. **C++ STL `sort()`** — now that you understand the internals (Quick+Heap+Insertion), learn to just use the built-in tool for real work.
11. **Custom Comparators** — essential for literally every “sort objects/pairs by some rule” interview question.
12. **Sorting + Two Pointers** — first major pattern combo; unlocks 2Sum/3Sum/4Sum family.
13. **Sorting + Greedy** — unlocks interval scheduling and resource allocation problems.
14. **Sorting + Intervals** — a hugely common interview category (Merge Intervals, Meeting Rooms, Insert Interval).
15. **Advanced sorting problems** (inversions, Kth element, Quickselect) — combines everything above into interview-hard problems.

This order moves from **simple mechanics** → **efficient general algorithms** → **specialized algorithms** → **applying sorting as a tool inside bigger patterns**, which mirrors how sorting actually shows up in real interviews: rarely as “implement quicksort,” almost always as “sort first, then solve the real problem.”